

2025 年全国大学生信息安全竞赛

作品报告



作品名称: DoTA: 面向字节码的轻量级智能合约分析框架

电子邮箱: gph04@qq.com

提交日期: 2025.08.13

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目录

摘要	1
第一章 介绍	2
第二章 背景	4
2.1 相关知识	4
2.2 相关工作	11
2.3 威胁模型	14
第三章 系统设计	16
3.1 系统设计概述	16
3.2 核心分析层	17
3.2.1 IR 优化	18
3.2.2 控制流图构建	19
3.2.3 程序样本切片	20
3.2.4 可达性分析器	22
3.2.5 模拟执行器	22
3.3 用户交互层	23
3.3.1 系统架构与核心功能总览	23
3.3.2 CFG 分析与模块化漏洞检测	25
3.3.3 LLM 增强引擎	27
第四章 合约 DoS 分类与检测挑战	29
4.1 调用回滚攻击 (CR)	30
4.2 算法复杂性攻击 (ACA)	32
4.3 GAS 耗尽 (GE)	34
4.4 悲伤攻击 (GA)	36
第五章 实验与评估	39
5.1 RQ1: DoTA 在检测智能合约漏洞方面具有怎样的有效性?	39
5.2 RQ2: 与当前主流工具相比, DoTA 在检测已知漏洞方面表现如何?	41
5.3 RQ3: DoTA 的特定组件在提升 CFG 生成精度方面是否不可或缺?	41
5.4 RQ4: DoTA 在分析大规模合约时具有怎样的可扩展性?	44
5.5 RQ5: DoTA 的动态验证模块是否不可或缺, 且能有效降低误报率?	46
第六章 总结与讨论	48
参考文献	54

摘要

随着区块链技术的迅猛发展及智能合约的广泛应用，智能合约安全问题已成为制约区块链生态系统可持续发展的关键瓶颈。尽管已有研究提出多种漏洞检测方案，但这些方法通常依赖于安全领域的专业知识，难以为非专业用户提供有效的风险评估；同时，大多数工具仅支持基于源码的自动化分析，而目前以太坊链上超过 99% 的合约并未公开源码，严重限制了此类方法的适用性与可扩展性。现有面向字节码的分析方法则普遍存在分析组件耦合严重、信息提取有限以及分析精度不足等问题。

为此，本文提出 DoTA——面向字节码的轻量级智能合约分析框架。DoTA 实现了首个支持 `hard_xref` 分析的字节码级 CFG，通过在中间表示（IR）优化后的控制流图上实施迭代反向切片算法，创新性地解决了延迟跳转语句的参数依赖难题，显著提升了控制流图构建的精度；同时，框架采用解耦式设计，为专业用户提供通用字节码分析接口，便于集成可插拔的漏洞检测器。进一步地，我们开发了融合检索增强生成（RAG）技术和 LLM Agent 辅助前端系统，为非专业用户提供友好高效的漏洞验证与风险评估能力。

为验证框架有效性，本文针对拒绝服务（DoS）漏洞开发了上层检测器。DoS 作为 OWASP 去中心化应用安全项目（DASP）TOP10 中的重要威胁之一，因其高破坏性和广泛影响而受到广泛关注。现有检测工具对 DoS 漏洞的识别局限于 `call-related` 与 `gas-related` 两类简单模式。本文构建了全面的 DoS 漏洞分类体系，涵盖调用回滚攻击（CR）、算法复杂性攻击（ACA）、GAS 耗尽攻击（GE）及悲伤攻击（GA）四大类，并建立代表性较好且可复用的数据集用于评估。

实验结果显示，DoTA 在 50 个已知漏洞合约中成功识别出 44 个，准确率达 88%，其中三类漏洞的准确率均为 100%；在 CR 漏洞检测精度方面较当前最优工具提升 31.03%。在对以太坊 15,949 份真实合约的大规模评估中，DoTA 检测出 342 份存在潜在 DoS 漏洞的合约。

关键字： 智能合约 轻量级程序分析框架 字节码 污点分析 LLM DoS

第一章 介绍

区块链技术的快速发展使其已成为数字经济时代的重要基础设施。根据 Fortune Business Insights 2025 年市场研究报告显示，全球区块链技术市场规模在 2024 年的价值为 201.6 亿美元，预计将从 2025 年的 311.8 亿美元增长到 2032 年的 39342 亿美元。[1] 以太坊作为最具影响力的智能合约平台，其 DeFi 生态的总锁仓价值 (TVL) 在 2021 年达到 1700 亿美元的历史峰值。然而，智能合约安全问题带来的经济损失同样触目惊心——年均因漏洞导致的资产损失超过 20 亿美元，其中拒绝服务 (DoS) 类漏洞因其破坏性强、影响范围广而备受关注。

由于 Web3 应用特有的前后端解耦架构，以及合约部署者可能选择不公开源代码的特性，使得面向字节码的分析工具成为安全检测的必备手段。尽管学术界和工业界已开发出多种检测工具 [2, 3, 4, 5, 6]，应用了包括模糊测试、符号执行和静态分析等先进技术，但在面向字节码分析领域仍存在显著不足。

为填补这一研究空白，本文提出 DoTA——面向字节码的轻量级智能合约分析框架。本工作的主要贡献与创新性包括：

1. **首个支持 `hard_xref` 分析的字节码级检测工具。**针对字节码分析中语法语义信息缺失的挑战，我们在中间表示 (IR) 优化后的控制流图上实施迭代反向切片算法，创新性地解决了延迟跳转语句的参数依赖分析难题。通过构建细粒度的控制流图，显著提升了分析精度。特别值得注意的是，本框架采用模块化设计，将底层静态分析与上层漏洞检测器完全解耦，这种架构在保证当前系统的检测精度的前提下，为后续研究提供了可扩展的分析平台，支持各类新型检测器的快速集成与验证。
2. **建立了完整的智能合约 DoS 漏洞分类体系与基准数据集。**通过系统梳理顶级会议论文、权威组织报告和世界级安全竞赛案例，我们整理出 76 种智能合约漏洞，并为 40 多种漏洞配备了可复现的 Docker 环境。由于 DoS 为 DASP TOP 10 的高危漏洞，并且当前的检测方法并不完备，因此我们聚焦 DoS 漏洞，将其细分为四大类：调用回滚、悲伤攻击、GAS 耗尽和算法复杂性攻击。为验证工具有效性，我们构建了包含良性/恶性样本的专用数据集，并在以太坊数万个块中的真实合约上进行了大规模测试。
3. **取得了显著的实践成果。**实验结果显示，DoTA 在 50 个已知漏洞合约中成功检测出 44 个，准确率达到 88%，并在在可横向比较的 CR 漏洞检测精度方面超越现有

SOTA 31.03%。基于负责任披露原则，我们通过漏洞赏金计划向多个项目方报告了发现的安全问题，在获得修复确认后将公布详细技术细节。这些成果充分证明了 DoTA 在实际应用中的价值。

4. **设计并实现了面向非专业用户的友好型网页前端系统。**本研究结合检索增强生成 (RAG) 与 AI 智能体 (Agent) 等技术，构建了一个由大语言模型 (LLM) 辅助的智能合约漏洞验证与风险评估前端系统。该系统在保持专业分析能力的同时，显著提升了对非专业用户的可用性与交互性，使用户能够在无需掌握底层安全知识的前提下理解并识别合约潜在风险，从而在预防合约诈骗、降低用户安全风险方面发挥了积极作用。

本文的后续组织如下：第 2 节介绍背景与相关研究工作；第 3 节详细阐述 DoS 漏洞的分类与特征；第 4 节深入讲解 DoTA 的技术实现；第 5 节展示实验评估结果；最后在第 6 节总结全文并展望未来研究方向。

第二章 背景

2.1 相关知识

2.1.1 区块链

定义与风险

区块链是一种去中心化、不可篡改的分布式账本技术，其核心是通过密码学与共识机制，在无需可信第三方的情况下实现数据的安全存储与验证。根据比特币白皮书，区块链是“一种完全通过点对点技术实现的电子现金系统” [7]，使得在线支付能够直接由一方发起并支付给另一方，中间不需要任何金融机构。以太坊黄皮书进一步拓展其内涵，提出区块链是“一个可编程的、支持图灵完备智能合约的分布式状态机” [8]。

区块链技术的应用场景广泛，涵盖金融、供应链、医疗、物联网等多个领域。以太坊作为智能合约平台，推动了去中心化金融（DeFi）、同质化代币（NFT）等新兴应用的发展。DeFi 通过智能合约实现去中心化交易、借贷和保险等金融服务，降低了传统金融的信任成本；NFT 则为数字艺术、游戏资产等提供了独特的所有权证明，推动了数字内容的创新与变现。区块链技术凭借去中心化、不可篡改等特性，已在金融、供应链、医疗等领域实现突破性应用。供应链管理中，IBM Food Trust 被沃尔玛用于食品溯源，将污染源定位时间从 7 天缩短至 2 秒；医疗健康领域，MedRec 系统允许患者通过私钥授权跨机构共享病历，提升数据互操作性。然而，区块链的开放性与不可逆性也带来严峻挑战：底层协议层面临 51% 攻击风险（如 2018 年比特币黄金遭攻击损失 1800 万美元），智能合约漏洞导致年均超 20 亿美元损失（Chainalysis 2023 数据），而隐私保护与监管合规的冲突（如 Tornado Cash 被制裁事件）凸显去中心化治理的复杂性。

以太坊

以太坊是区块链技术发展的重要里程碑，作为第一个图灵完备的区块链平台，它突破了比特币的价值转移限制，为去中心化应用提供了全新基础设施。以太坊由布塔林于 2013 年提出，其核心创新在于实现了可编程的区块链。与比特币仅支持简单脚本的设计不同，以太坊引入了图灵完备的智能合约系统，允许开发者部署任意复杂的计算逻辑。

以太坊的核心组件包括：作为“世界计算机”的以太坊虚拟机 (EVM)，这是一个基于堆栈架构的隔离沙盒环境，拥有 140 多个操作码，执行智能合约字节码并确保所

有节点结果一致；区别于比特币 UTXO 的账户模型，分为用户控制的外部拥有账户 (EOA) 和存储代码与状态的合约账户；防止资源滥用的 Gas 机制，为每个操作定价并要求用户支付费用 (2021 年 EIP-1559 后引入通缩机制)；以及状态转换系统，通过区块中的交易集合将全局状态从一个版本转换到下一个，并使用 Merkle Patricia Trie 存储结构确保数据完整性和验证效率。

此外，以太坊还催生了丰富的区块链应用生态，其中去中心化金融 (DeFi) 通过 Uniswap 等项目实现了万亿美元的交易量；同质化代币 (NFT) 如 Bored Ape Yacht Club 则引爆了数字艺术与收藏品市场；去中心化自治组织 (DAO) 为社区共治提供了新模式；而跨链桥技术也正通过 Wormhole 等项目连接着日益发展的多链世界。

2.1.2 智能合约

智能合约基础概念

智能合约是运行在区块链网络上的自动执行程序，由代码和数据组成，一旦部署便能自主运行并强制执行约定的规则。智能合约本质上是一种特殊的账户，与用户控制的外部账户不同，它由代码控制且能在满足特定条件时自动执行，无需人为干预。

智能合约的核心特性包括去中心化、确定性、透明性和不可篡改性。去中心化体现在其执行不依赖单一中心化机构；确定性指相同输入必然产生相同输出；透明性使得所有代码和交易对网络参与者可见；不可篡改性确保一旦部署便无法修改，从而保证执行的可靠性。

智能合约与传统软件的根本区别在于其执行环境和不可变性。传统软件运行在中心化服务器上，可随时更新修改；而智能合约部署后代码不可更改，任何状态变更都通过交易触发，并由整个网络共同验证，这种机制为数字协议的可信执行提供了技术保障。

智能合约底层字节码 (Bytecode) 机制

智能合约的底层字节码是理解其工作原理的关键，它是高级编程语言编译后的底层表示，直接被区块链虚拟机解释执行。以以太坊为例，Solidity 等高级语言编写的智能合约通过编译器转换为 EVM 字节码，这种字节码是一系列十六进制数字，每个数字代表特定的操作码 (Opcode) 或操作数。

EVM 字节码包含多种操作码类型，涵盖算术运算 (ADD、SUB、MUL)、逻辑运算 (AND、OR、XOR)、栈操作 (PUSH、POP、DUP)、存储操作 (SLOAD、STORE)、流程控制 (JUMP、JUMPI)、环境信息获取 (CALLER、CALLVALUE) 等功能。这些操作码共同构成了 EVM 的指令集，支持智能合约执行各种复杂逻辑。其设计考虑了

Gas 计费机制，不同操作码消耗不同数量的 Gas。智能合约字节码文件通常包含三部分：初始化代码、运行时代码和元数据。初始化代码仅在合约创建时执行一次，用于设置初始状态并返回运行时代码；运行时代码则是合约实际功能的实现，在每次合约调用时执行；元数据保存了编译器版本等字节码信息。

智能合约的执行环境与运行机制

智能合约的执行依赖于一个由区块链提供的、完全隔离且确定性的虚拟机（Virtual Machine）环境，其中以太坊虚拟机（EVM）最具代表性。这个沙盒环境确保了同一份合约代码在任何节点上执行都会产生完全相同的结果。合约的生命周期始于开发者用 Solidity 等高级语言编写代码，随后将其编译成虚拟机可读的字节码，并通过一笔交易部署到区块链上，从而获得一个永久地址。部署后，合约便进入待命状态，等待用户或其他合约通过发送一笔包含特定函数调用和数据的交易来触发执行。其核心运行机制由 Gas 驱动，这是一个为计算资源定价的系统，合约中的每一步操作都有明确的 Gas 成本。用户必须支付 Gas 费用以激励验证者处理交易，并防止恶意代码耗尽网络资源。一旦合约执行完毕，其引发的状态变更（如账户余额的增减）会被打包进区块，并通过网络的共识协议（如 PoS）进行验证。只有当全网达成共识后，这个区块才会被永久添加到链上，使合约的执行结果变得不可篡改，成为全网公认的既定事实。

2.1.3 智能合约安全

智能合约安全问题的重要性与日俱增，其原因在于它们已成为承载巨额资金的核心基础设施。智能合约的不可篡改性是一把双刃剑：它保证了规则的稳定执行，但也意味着代码中的任何漏洞一旦部署便永久存在，难以修复。同时，随着 DeFi、NFT 等应用场景的复杂性不断增加，引入潜在安全缺陷的风险也随之升高。

这些风险的产生，往往与开发者安全意识参差不齐以及编程语言本身的特性有关。以主流语言 Solidity 为例，其强大的低级调用等功能虽为开发带来灵活性，但若使用不当，极易引发严重漏洞。业界总结的 DASP TOP 10 [9] 就列举了多种常见安全漏洞，如重入攻击（Re-entrancy）、整数溢出（Integer Overflow）、未初始化存储变量和未授权调用等。这些漏洞一旦被利用，可能导致灾难性的资金被盗或合约失能，不仅威胁着用户的个人资产，也对整个区块链生态的稳定性构成了深远挑战。

更多的示例可以参考 DeFiHackLabs 提供的漏洞案例列表 [10]。

综上所述，智能合约的安全问题亟待解决。为了应对这些挑战，研究人员和开发者不断改进智能合约的开发工具和安全分析方法，以减少漏洞的发生并降低其带来

表 2.1 近年部分智能合约安全事件

时间	项目名称	漏洞类型	损失金额
2025 年 3 月 18 日	DCFToken	交易滑点保护缺失	42.8 万美元
2025 年 3 月 16 日	wKeyDAO	函数权限控制缺失	7.3 万美元
2025 年 3 月 14 日	H2O 项目	弱随机数生成机制	2.247 万美元
2024 年 12 月	CloberDEX	经典重入漏洞	50.1 万美元
2020 年	bZx 平台	闪电贷操纵价格预言机	900 万美元
2017 年	Parity 钱包	初始化代码设计缺陷	1.5 亿美元
2016 年	The DAO	重入漏洞	5000 万美元

的风险。

J. Feist 等人提出了 **Slither**[11]，这是一种静态分析工具，专为 Solidity 智能合约设计。它通过构建抽象语法树（AST）和控制流图（CFG）来分析代码，并检测常见的安全漏洞，如重入攻击、未初始化变量等。J. F. Ferreira 等人提出了 **SmartBugs**[12]，这是一个基于插件的框架，集成了多种分析工具（如 Slither），支持静态分析和动态分析。它通过统一的接口调用不同的分析工具，从而实现对智能合约的全面检测。CONSENSYS 组织开发了 **Mythril**[13]，这是一种基于符号执行的智能合约分析工具。它通过模拟 EVM 的执行过程，探索所有可能的执行路径，从而发现潜在的漏洞，如整数溢出、未授权访问等。而 Loi Luu 等人提出 **Oyente**[14]，这是最早提出的智能合约安全分析工具之一，采用符号执行和污点分析相结合的方法。它通过分析智能合约的字节码，检测常见漏洞，如重入攻击和时间依赖性问题。

上述工具在智能合约安全分析领域发挥了重要作用，为开发者提供了强大的漏洞检测能力。然而，这些工具也存在一定的局限性，例如符号执行工具可能面临路径爆炸问题，静态分析工具可能产生误报或漏报。因此，结合多种分析方法的混合工具逐渐成为研究热点。

2.1.4 DoS 漏洞

拒绝服务（Denial of Service, DoS）漏洞是一种通过耗尽目标系统资源（如计算能力、内存、网络带宽等）或阻止合法用户访问服务的攻击方式。攻击者通常通过发送大量请求、触发高复杂度计算或利用系统设计缺陷，使目标系统无法正常运行。DoS 攻击的目标不仅限于单一系统，还可能扩展到整个网络或分布式系统，其影响范围和

危害程度因攻击方式和目标系统的不同而有所差异。

在不同的应用场景中，DoS 漏洞可能表现为多种形式。例如，在传统 Web 应用中，攻击者可能通过发送大量恶意请求使服务器过载，导致合法用户无法访问服务，甚至可能导致服务器崩溃。在区块链系统中，攻击者可能通过发送大量交易或触发高复杂度的智能合约操作来耗尽区块链网络的计算资源，从而阻止其他用户的交易被正常处理。此外，在物联网设备中，由于设备通常计算能力较低，攻击者可能通过简单的请求使设备陷入瘫痪，导致其无法正常运行。这些漏洞的危害体现在多个方面，包括服务中断、经济损失、用户信任下降以及系统安全性的整体削弱，甚至可能对整个生态系统的稳定性产生深远影响。

在智能合约中，DoS 漏洞尤为突出。作为 OWASP[15] 排名前十的漏洞之一，其他类型的应用程序或许可以通过 patch 的方式修复，而智能合约只需一次此类攻击就可能永远下线。例如，攻击者可以通过消耗合约的 GAS 限额来阻止其他用户的交易执行，或者利用复杂的算法操作使合约陷入无限循环。此外，DoS 漏洞还可能通过阻塞关键合约功能，导致依赖该合约的其他合约或应用程序无法正常运行，从而引发连锁反应，进一步扩大攻击的影响范围。因此，全面识别和检测智能合约中的 DoS 漏洞已成为当前研究的重要课题。

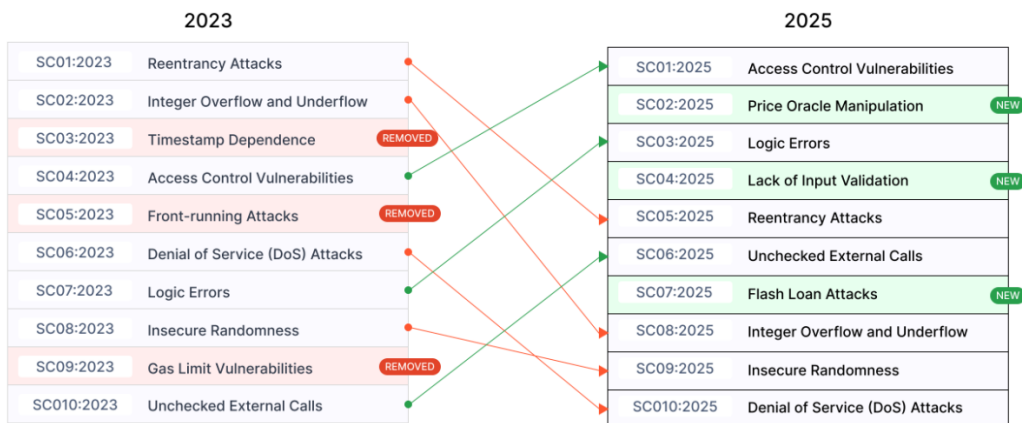


图 2.1 2023-2025 年 OWASP 榜单

Asem Ghaleb 等人提出了 **Etainter**[16], 这是一种基于污点分析的智能合约漏洞检测工具，专注于 call-revert 和 GAS 耗尽漏洞。它通过构建智能合约的控制流图 (CFG) 和数据流图 (DFG)，分析变量的污点传播路径，从而识别潜在的漏洞点。J. Xu 等人提出了 **DoSChecker**[17], 这是一种专为检测智能合约中 DoS 漏洞设计的分析工具。它通过符号执行技术模拟智能合约在以太坊虚拟机 (EVM) 中的执行过程，探索所有可能的执行路径，从而发现潜在的漏洞。

然而，现有工作对 DoS 的分类往往过于狭隘，仅局限于 call related 和 gas related 两种模式，而忽略了其他类型的 DoS 漏洞。这种局限性导致现有检测工具和方法无法全面覆盖智能合约中的所有 DoS 漏洞类型，从而增加了漏洞被攻击者利用的风险。

在本工作中，我们分析了数百份影响力广泛且含有漏洞的智能合约，总结整理了 76 类漏洞，并从中提取了四类典型的 DoS 漏洞，包括调用回滚攻击（Call Revert Attacks, CR）、算法复杂性攻击（Algorithmic Complexity Attacks, ACA）、GAS 耗尽（Gas Exhaustion, GE）和悲伤攻击（Griefing Attacks, GA）。这四类漏洞涵盖了智能合约中绝大多数的应用场景，且危害性较大。通过对这些漏洞的深入研究，我们不仅揭示了其潜在的攻击方式和影响，还为智能合约的安全检测和防护提供了新的思路和方法。

2.1.5 LLM 相关

ReAct 范式与 AI 智能体 ReAct [18] 范式是一种将推理（Reasoning）与行动（Acting）紧密结合的智能体设计方法，旨在提升 AI 系统解决复杂任务的能力。该范式通过让智能体在执行过程中不断交替进行“思考-行动-反馈”循环，使其能够边推理边决策，动态调整策略，而非简单地将推理与行动分开处理。具体来说，智能体在面对问题时，先基于已有信息进行推理，形成初步判断或查询，然后执行相应的动作（例如调用外部工具、检索知识或操作环境），再根据行动结果更新认知状态，继续下一轮推理和行动。这样一来，智能体能够更好地处理多步骤、多信息源融合的问题，显著提高任务完成的准确性和效率。

AI 智能体（Agent）是一类具备自主感知、推理决策和行为执行能力的智能系统。它们能够主动感知环境状态，基于目标和当前信息进行推理分析，制定并执行相应的行动计划，进而实现动态环境下的自主任务完成。智能体不仅能够响应外部输入，还能利用内在的知识库和推理机制进行复杂判断，支持连续、交互式的任务处理。结合 ReAct 框架的智能体，能够更灵活地适应变化的环境，通过反复思考与行动的闭环机制，实现高效、智能的自主决策和执行，广泛应用于自然语言处理、机器人控制、自动化推理等领域。

表 2.2 Agent 与传统大型语言模型 (LLM) 对比

对比维度	传统大型语言模型 (LLM)	智能体 (Agent)
核心功能	以生成自然语言文本为主，回答问题、生成内容。	集成感知、推理、计划与执行能力，能够自主完成复杂任务。
推理能力	基于训练数据的模式匹配与统计推断，推理较为隐式且有限。	结合显式推理机制和多轮交互，支持多步骤复杂推理过程。
行动能力	通常不具备主动执行操作的能力，只能生成文本响应。	具备主动执行动作的能力（调用工具、运行代码、操作环境等）。
任务处理方式	以单轮或有限多轮对话形式提供静态答案。	通过循环的“感知-推理-行动-反馈”机制，动态调整策略完成任务。
多模态集成能力	主要处理文本输入和输出，少数支持有限多模态。	容易集成多模态感知（视觉、语音等）和多源信息融合。
灵活性与扩展性	依赖于预训练数据和模型设计，扩展外部能力受限。	通过插件、代码执行、外部工具调用等方式灵活扩展功能。

RAG 检索增强生成 检索增强生成 (Retrieval-Augmented Generation, RAG) 是一种结合外部知识检索与大型语言模型 (LLM) 生成能力的技术框架，旨在增强 LLM 的准确性、时效性与上下文感知。RAG 通过在生成前动态检索外部知识并注入模型，有效缓解幻觉、知识过时和缺乏领域特异性等问题，并提升透明度。Meta 于 2020 年首次提出 RAG，其核心理念是知识外部化，使模型基于可检索信息进行推理，实现知识的快速更新与灵活适配。

RAG 的关键价值在于降低生成幻觉、增强事实性、支持知识溯源，并以低成本实现专业化。相较于微调 LLM，RAG 通过构建领域知识库赋予通用模型专业能力，且不依赖昂贵的数据准备与训练资源。

RAG 工作流程分为摄入与推理两阶段。摄入阶段构建结构化知识库，通过文档收集、预处理与语义分块，并嵌入为向量存入数据库；推理阶段将用户查询转为向量，检索相关文本块，与原始问题组合形成增强提示，供 LLM 生成响应。文档质量、分块策略与嵌入模型表现直接影响最终效果。

RAG 为 LLM 应用提供实时性、透明度、专业化与高可维护性的架构方案，提升生成质量与用户信任，并显著降低系统扩展和更新成本，是构建可信赖 AI 系统的重要基础。

表 2.3 RAG 与标准 LLM 对比

特性	RAG	标准 LLM
知识来源	预训练知识 + 实时外部知识	仅预训练知识
响应生成基础	预训练知识 + 检索到的上下文	仅预训练知识
处理新/近期信息	动态检索	受训练截止日期限制
事实准确性/幻觉	更高, 减少幻觉	易产生幻觉
领域特异性	知识库轻松适应	需微调/重训练
透明度/可追溯性	高 (可引用来源)	低 (不透明)
知识更新成本	较低 (更新知识库)	较高 (重训练模型)

2.2 相关工作

如前文所述, 自动化检测智能合约中的漏洞是一个重要的研究方向, 而许多研究者和开发者已经提出了多种方法和工具来应对这一挑战。以下是一些主要的研究方向和工具:

2.2.1 动态分析技术

模糊测试技术研究

模糊测试 (Fuzzing) 作为一种高效的动态自动化测试技术, 通过向目标程序注入大量随机或半随机生成的异常输入数据, 系统地触发程序异常行为, 从而有效识别潜在的安全漏洞。与传统软件测试相比, 模糊测试具有自动化程度高、误报率低等显著优势, 这使其成为智能合约安全检测领域的重要方法 [19]。在智能合约应用场景下, 模糊测试需要特别考虑合约的状态依赖性、环境参数配置以及复杂交易序列等关键因素, 其技术核心在于如何生成符合合约业务逻辑的有效输入数据, 并在测试执行过程中实现输入策略的动态优化, 以提升漏洞检测效率。

现有研究主要围绕三类模糊测试方法展开: 基于生成的模糊测试、基于变异的模糊测试以及混合测试方法。基于生成的模糊测试 (如 **ContractFuzzer**[5]) 能够快速覆盖基础接口, 但在处理复杂状态转换场景时表现欠佳。基于变异的模糊测试 (如 **sFuzz**[20]) 通过对已有测试用例进行智能变异, 能够发现更深层次的漏洞, 但其检测效果高度依赖于初始种子质量。为克服单一技术的局限性, 研究者提出了结合符号执行与模糊测试的混合方法 (如 **Smartian**[6]), 该方法通过符号执行解决复杂路径约束问题, 再利用模糊测试扩大测试覆盖范围, 实现了检测深度与广度的平衡。

近期的大规模基准测试 (涉及 2000 个智能合约样本) 揭示了各测试工具的性能差异: 在吞吐量方面, **EFCF**[21] 以每秒 287 个事务的处理速度领先, 而 **SmartGift**[22]

仅为每秒 0.07 个事务，研究表明采用轻量级 EVM 实现的工具普遍具有更高吞吐量；在漏洞检测能力方面，现有工具对 9 类常见漏洞的平均检测率为 53%，其中整数溢出漏洞检测率最高 (68%)，权限漏洞检测率最低 (32%)；在代码覆盖率方面，领先工具 (如 **ityFuzz**[23]) 的基本块覆盖率可达 82%，但路径覆盖率普遍低于 50%，这表明现有方法对复杂控制流的处理能力仍有待提升。值得注意的是，模糊测试技术仍面临若干挑战：对复杂逻辑和状态的覆盖能力有限可能导致漏报或误报，对函数 ABI 的硬性需求 (这意味着对源码的高依赖) 且通常需要消耗大量计算资源和测试时间。

符号执行技术研究

符号执行技术通过将程序变量抽象为符号值并构建路径约束条件，实现了静态分析与形式化验证的优势融合。在智能合约安全分析领域，**Mythril**[13] 是最具代表性的符号执行工具之一，该工具通过符号化建模 EVM 字节码执行过程，系统探索所有可能的状态空间，并利用 SMT 求解器验证安全属性违例情况，可有效检测重入漏洞、整数溢出漏洞及不安全自毁操作等安全问题。尽管面临路径爆炸问题的挑战，符号执行技术因其深度状态探索能力，仍然是发现复杂逻辑漏洞的重要方法。

为提升特定类型漏洞的检测效果，研究者提出了多种改进方案：**Etainter**[16] 通过整合符号执行与污点分析技术，专门针对 gas 相关漏洞进行检测，该工具通过建模字节码的污点传播过程，成功识别出可能导致 gas 耗尽的危险执行路径 (如无限循环和循环调用等)，在 60,612 份真实合约测试中发现了 2,763 个潜在漏洞；**CRUSH**[24] 则专注于存储碰撞漏洞检测，通过建立存储槽使用模型并分析变量间的潜在冲突，有效识别了以太坊合约中的隐蔽存储安全问题。

然而，符号执行技术仍存在明显局限性：路径爆炸问题导致分析效率低下，计算资源消耗过大，这些因素都制约了该技术在大型复杂合约分析中的应用效果。

2.2.2 静态分析技术

静态分析是检测智能合约漏洞最广泛使用的技术之一，通过对源代码或字节码进行结构化分析，无需实际执行合约即可识别潜在安全问题。与动态分析不同，静态分析在部署前就能发现漏洞，极大降低了安全风险和修复成本。基于其工作原理，静态分析可分为基于模式匹配和基于语义分析的方法。

基于模式匹配的静态分析

该方法通过构建完善的漏洞特征库，系统性地识别代码中与已知漏洞模式相匹配的结构特征。代表性工具 **Slither**[11] 采用多层次的程序分析技术，包括精确的抽象语法树 (AST) 构建、控制流图 (CFG) 生成以及自定义中间表示 SlithIR 的转换分析，

实现了对 70 余种智能合约漏洞的高效检测。实验数据表明，Slither 在保持低误报率 (4.7%) 的同时，能够准确识别重入攻击、访问控制缺陷和算术溢出等常见漏洞，其卓越的性能使其成为工业界事实上的标准分析工具。

基于语义和数据流的静态分析

此类方法关注变量之间的依赖关系和信息流动，更适合检测复杂的漏洞类型。**Ethainter**[3] 通过信息流分析框架实现了对复合漏洞的检测，该工具引入“污点”和“清洗剂”概念，追踪敏感数据的传播路径，特别适合检测未受保护的 `selfdestruct` 和 `delegateCall` 等高危函数调用。实验表明，Ethainter 能分析 75.8% 的以太坊合约，且对比手动审计有 94.6% 的准确率。**RNVulDet**[2] 专注于随机数生态系统的安全分析，通过构建异质化风险传播图和全局污点传播分析，对合约中随机数生成和使用的模式进行全面评估。该工具成功揭示了 44,264 笔攻击交易，强调了伪随机数机制在区块链环境中的固有缺陷。**AVVERIFIER**[4] 针对地址验证漏洞设计了轻量级静态分析器，通过模拟 EVM 操作码执行并定义精确的漏洞特征，在超过五百万份合约中检测出 8,12 个先前未发现的地址验证漏洞。该工具将字节码反汇编为操作码序列，并结合数据流分析追踪地址参数的验证逻辑。

静态分析的优势与局限

静态分析工具执行效率高，可在毫秒到秒级别完成合约分析，适合持续集成环境；其不需要部署合约即可进行验证，显著降低安全成本。然而，静态分析也存在固有局限：对跨合约复杂交互的建模能力有限，难以完全模拟外部合约交互。此外，由于 EVM 的复杂性和 Solidity 语言特性，某些动态特性（如时序依赖问题）的检测效果有待提升。这些局限性正是本研究重点突破的方向，我们通过创新性地结合多种静态分析技术，在保持高效检测的同时，显著提升了分析的深度和广度。

2.2.3 大语言模型

智能合约漏洞检测正经历从传统规则匹配向大语言模型 (LLM) 驱动的“认知推理”范式转变。传统静态分析工具（如 Slither、Mythril）依赖预定义规则，虽能快速扫描代码，但对复杂逻辑漏洞（如重入攻击、权限控制缺陷）的检测能力有限。动态分析（如符号执行工具 Oyente、模糊测试工具 ContractFuzzer）通过模拟执行环境提升准确性，但面临计算成本高、难以覆盖长代码路径的瓶颈。早期直接应用通用 LLM（如 ChatGPT）虽展现语义理解潜力，却受限于输出不稳定、长合约解析偏差及解释模糊等问题，难以满足工业级检测需求。

大语言模型通过多模态融合、领域强化训练和动态架构优化显著提升检测能力。

例如，多模态框架 **Agent4Vul**[25] 将合约字节码转换为控制流图（CFG），结合图注意力网络（GAT）提取拓扑特征，同时利用 LLM 生成带漏洞推理的代码注释，实现语义与结构信息的协同分析，使重入漏洞检测 F1 分数提升至 92.53–98.58%。领域优化模型如 **Smart-LLaMA-DPO**[26]，通过持续预训练注入合约语法知识，并采用直接偏好优化（DPO）策略结合工程师反馈微调，将误报率降低 6.18%，解释可信度达 81.15%。动态架构如双视角智能体（审计者 + 攻击者视角协同）和 MOS 框架（专家路由网络），则通过模拟业务逻辑与攻击路径，覆盖传统工具盲区，提升复杂漏洞的定位精度。

尽管技术进步显著，LLM 驱动的检测仍面临轻量化部署、跨链兼容性和动态更新机制三大挑战。工业场景中，模型需通过剪枝量化适应链上低延迟需求；跨链支持需兼容 EVM、WASM 等多虚拟机环境，解决生态碎片化问题；而动态更新则依赖漏洞-修复案例库的持续构建，以应对新型攻击模式。未来方向指向多智能体协同框架，整合审计、攻击模拟与修复生成角色，实现智能合约全生命周期安全治理。当前技术已证明 LLM 能从“辅助工具”升级为“自主化安全代理”，但其规模化落地仍需在精准性、可解释性与适应性间寻求更优平衡。

2.3 威胁模型

在智能合约层面，DoS 攻击的核心目标是使合约在功能层面无法按预期响应正常用户的交互请求，或导致某些功能处于不可用状态。为更系统地刻画智能合约中 DoS 漏洞的攻击面，我们将潜在的攻击者划分为以下两类：恶意调用者（Malicious Callers）与恶意部署者（Malicious Deployers），并分别分析其可利用的攻击路径及潜在危害，如图2.2所示。

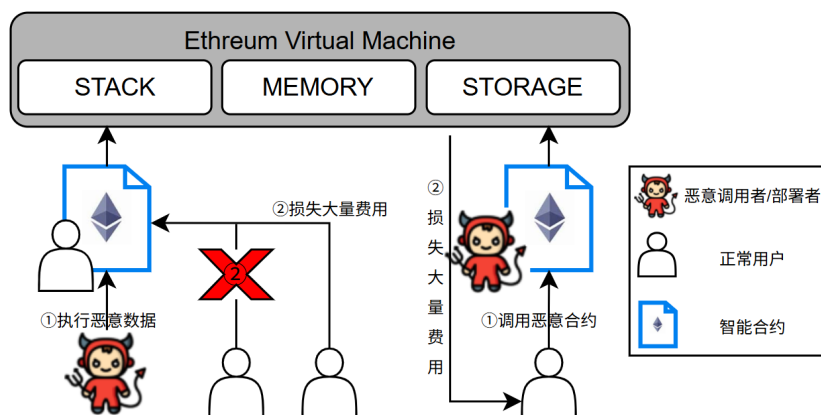


图 2.2 面向 DoS 的智能合约威胁模型

2.3.1 恶意调用者

恶意调用者指的是在系统运行过程中，以外部用户身份与合约交互的攻击者。他们的攻击目标通常是通过精心构造的输入数据、调用路径或交易序列，诱导合约触发高计算复杂度、边界异常或资源竞争等情形，造成服务不可用、关键逻辑无法执行或 Gas 耗尽等问题。在治理合约等依赖提案流程的系统中，恶意调用者可能提交计算开销巨大的提案，例如嵌套循环、冗余状态检查、复杂投票逻辑等。一旦该类提案被通过并执行，不仅会严重拖慢治理进程，甚至可能因单次调用消耗过多 Gas 而使得整个执行流程在链上失败，进而阻断关键治理操作，造成链上治理流程被“冻结”。此外，攻击者还可以利用某些函数在合约中缺乏 Gas 限制的特性，通过批量或重复调用特定接口，实现对合约状态的锁定、队列阻塞、资源饱和等攻击效果。

2.3.2 恶意部署者

恶意部署者是指以合约发布者的身份，故意编写或部署含有隐蔽拒绝服务逻辑的合约，其核心手法是在表面上提供正常交互接口和前端交互逻辑，实际在字节码层面或函数实现中嵌入高复杂度计算、无限循环触发点、Gas 陷阱等构造。这类攻击具有较强的隐蔽性，主要得益于当前 Web3 体系中前后端高度解耦：普通用户通常仅通过前端页面与合约交互，而无法或未能对合约字节码或 ABI 逻辑进行深入验证。因此，恶意部署者可以通过构造诱导性强的前端页面，引导用户与实际存在 DoS 陷阱的合约交互，导致 Gas 大量损耗，甚至造成链上资金损失或应用级拒绝服务。

典型的攻击形式包括：在无明确 Gas 上限的合约函数中嵌入复杂度不可控的迭代逻辑；利用函数嵌套、回调或逻辑分支结构触发 Gas 爆炸；构造状态变量依赖结构，使用户每次调用的计算量随时间/次数增长；在数据结构中插入“黑洞”节点，如无限遍历或排序死循环。

此类威胁进一步强化了对字节码级分析能力的依赖，尤其是在源码未公开的情况下，若缺乏静态分析、反编译或行为建模能力，将极难识别潜藏的 DoS 攻击点。

第三章 系统设计

3.1 系统设计概述

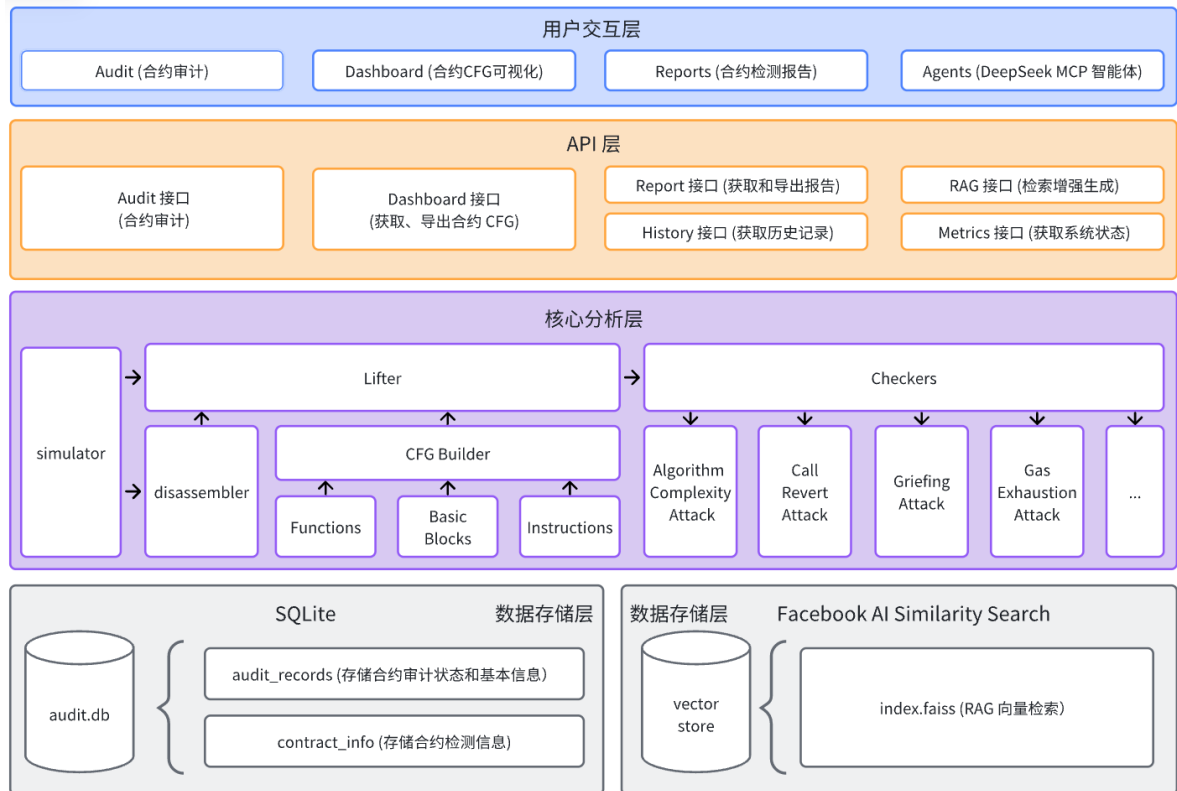


图 3.1 项目整体架构

鉴于智能合约在区块链生态中的广泛应用及其安全性对用户资产和系统稳定性的重要影响，本系统不仅为安全专家提供了解耦的字节码分析结果接口，还旨在为非专业用户提供一套便捷且高效的智能合约安全隐患检测解决方案。为此，我们设计并实现了完整的前后端交互架构，使用户能够方便地提交智能合约的字节码或源码，触发自动化漏洞分析流程。本系统设计充分考虑了智能合约安全检测的实用性与扩展性，支持部署于高性能服务器环境中，以满足多用户并发访问及大规模合约分析的需求，力求为区块链社区提供可信赖且易用的安全检测服务。

系统整体架构如图3.1所示，主要包括用户交互层、API 服务层、核心分析引擎及数据存储层。用户通过友好的前端界面与系统进行交互，提交待检测的合约信息，系统则通过对接 DoTA 的 API 接口，完成对合约的静态及动态安全分析。分析结果会被及时返回给用户，帮助其直观地识别潜在的安全风险。其中，核心分析层承担合约字节码及源码的深度解析、程序分析及漏洞检测，其具体实现原理及技术细节将在 §3.2

中详细介绍。前端交互设计以及数据管理与存储方案则将在 §3.3 中进行全面阐述。

3.2 核心分析层

核心分析层是整个系统的技术基础，负责对智能合约字节码进行静态分析，并进一步检测分析结果，提取其中的漏洞。其目标是将原始的低层次字节码还原为具有语义结构的信息图谱，并基于该语义支撑进行漏洞检测。

首先，DoTA 会对输入的字节码进行预处理。具体而言，通过正则模式匹配方式，剔除 constructor code 与 metadata 部分，我们仅保留合约的 runtime code 作为后续分析的基础。随后，DoTA 将提取到的运行时代码反编译为 Yul 汇编语言，再进一步提升至中间表示（Intermediate Representation, IR），以便构建控制流图并进行更高层次的静态分析，相关细节将在下一小节详细描述。在获取控制流图之后，DoTA 会使用 networkx 库来获取程序当中的循环。

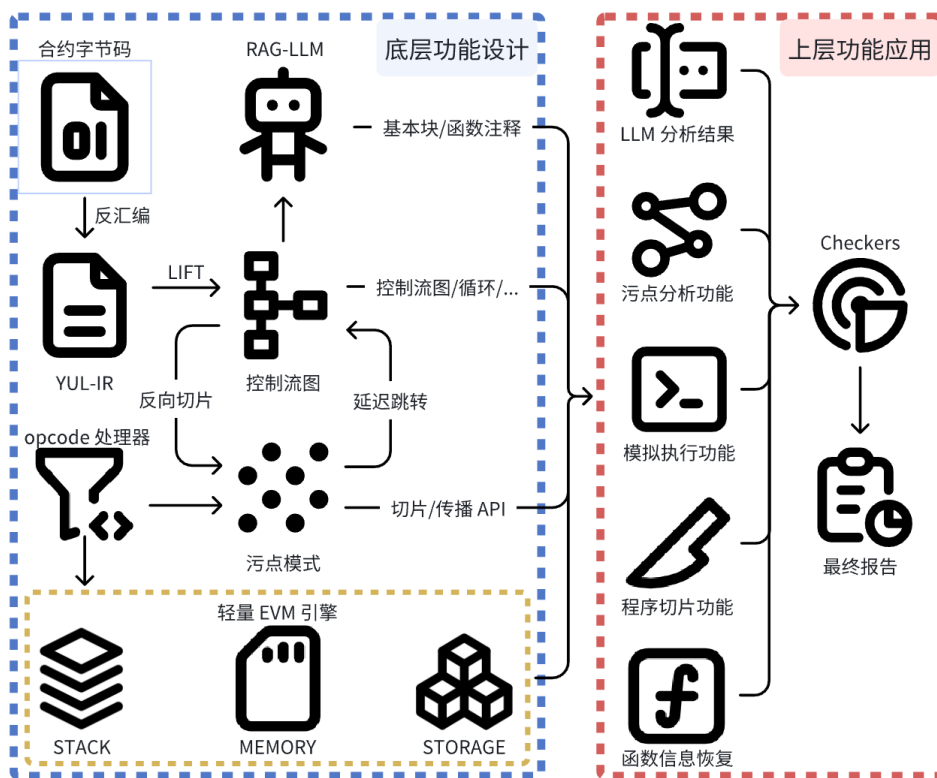


图 3.2 核心分析层架构

如图3.2所示，核心分析层为上层漏洞检测器提供了一系列结构化、语义丰富的分析结果与功能接口，将原始字节码转化为可供深度分析的程序表示。这些分析结果是实现精准、高效漏洞检测的基础。

3.2.1 IR 优化

在本作品中，我们首先通过模式匹配完成**常量传播**，将常量在指令中尽可能提前传播，比如将 `PUSH [a]`、`PUSH [b]`、`ADD` 替换为 `PUSH [a+b]`；然后**消除 dead code**，solidity 编译器在编译过程中经常会加入一些没有实际用处的平栈指令，我们通过反向切片寻找某些 `pop` 指令的污染源，如果他们全为 `push [const]`，我们则将这一对 `push pop` 删除，同时将切片中涉及到栈上相关位置的指令略作修改，比如将 `DUP`、`SWAP` 的操作数进行修复等。通过 IR 优化，我们可以得到一个尽可能精简的控制流图。

延迟跳转语句/hard_xref

在 EVM 字节码分析中，延迟跳转语句是指一类特殊的控制流转移指令（`JUMP` 或 `JUMPI` 指令），其跳转目标地址并非在执行跳转指令时直接从栈顶获取（通过紧邻的前一条 `PUSH` 指令压入），而是需要通过一系列中间计算或内存/存储访问间接推导得出。这类跳转指令在控制流图构建过程中构成了重大挑战，因为它们引入了动态性，使得静态分析无法直接确定其跳转目标。

核心特征与挑战 首先，跳转目标地址未以 `PUSH <immediate_value>` 的形式出现在跳转指令之前。目标地址的计算过程可能涉及从内存（`MLOAD`）或存储（`SLOAD`）中加载，通过算术/逻辑运算（`ADD`、`SUB`、`AND`、`OR`、`XOR`、`MUL`、`DIV`、`MOD` 等）计算得出，通过调用数据（`CALLDATALOAD`）或返回数据（`RETURNDATALOAD`）获取，以及复杂的栈操作组合（如 `DUP`、`SWAP` 后接运算）。其次，目标地址的值依赖于程序执行时的具体状态（如合约存储内容、调用输入数据、内存状态等），在静态分析阶段无法直接确定其具体值。

重要性 成功解析和处理延迟跳转语句对于构建准确且完整的 CFG 至关重要。它使得分析工具能够正确识别函数调用（通过 `CALL`/`STATICCALL`/`DELEGATECALL` 后的跳转或通过函数选择器分派到内部函数入口）和跳转表（`JUMPTABLE`）的实现。发现潜在的、通过复杂计算或外部输入控制的跳转路径，这些路径可能被用于恶意行为（如后门）或包含关键逻辑。为后续的数据流分析（如污点分析、符号执行）、漏洞检测（如重入、未检查调用）和路径可达性分析提供更可靠的基础控制流结构。辅助识别合约的内部函数划分、库调用、代理模式等高级结构。因此，“延迟跳转语句”的处理（主要通过反向切片技术实现）是克服 EVM 字节码静态分析中动态性障碍的关键环节，是构建高精度 CFG 和实现深度智能合约安全分析的基石。

3.2.2 控制流图构建

控制流图（CFG）是静态分析中的核心结构，用于刻画程序的执行路径。其构建过程如下：

构建指令块（Instructions） 在获得 IR 之后，DoTA 首先按照顺序遍历指令列表，从第一条指令开始进行分组，构建初始的指令块（Instruction Blocks）。每当遇到一条满足以下任一条件的指令时，即认为其为一个新块的开始：其一，该指令是整个指令序列的第一条指令；其二，该指令为 JUMPDEST；其三，其前一条指令为 JUMPI、JUMP、STOP 或 RETURN。在遍历结束后，若存在未被归入任一块的残余指令序列，则将其作为一个独立的指令块，以确保覆盖整个程序空间。

构建基本块（Basic Blocks） 在初始指令块的基础上，进一步初始化为基本块，每个基本块内部的指令构成原子性的执行单元。此阶段形成初始的控制流图节点集合，为下一步跳转边的构建提供支撑。

构建 CFG 的边（CFG Edges）

1. 简单边的构建及前后驱节点标注 控制流图的边主要由跳转语义决定，核心处理两类跳转指令：JUMPI 和 JUMP。对于 JUMPI，需提取两个目标：条件跳转目标（jump_target）和顺序执行目标（fallthrough_target），其中，条件跳转目标通常由其前一条 PUSH 指令提供，表示在条件成立时跳转至的地址；顺序执行目标表示条件不成立时的顺序执行路径，即当前块的下一基本块。对于 JUMP，仅需解析跳转目标，亦由其前一条 PUSH 指令决定。若无法从前一条指令中解析出跳转地址，则标记为 hard_xref，交由后续模块进一步解析。完成跳转目标的解析后，系统建立当前基本块与其目标块之间的控制流边，并标注其前驱与后继关系；若某基本块无任何后继节点，则标识为终止块（Terminal Block）。

2. Hard_xref 跳转修复 在某些动态跳转场景中，目标地址并不以显式的 PUSH 指令呈现，形成 hard_xref。为修复此类跳转，我们采用反向切片分析（Backward Slicing）技术，反向寻找到未解决的 JUMP/JUMPI 指令（即延迟跳转语句）所消耗的栈上参数是由哪条指令入栈的，从而找到延迟跳转语句的参数。反向切片的实现细节将在下一小节详细描述。

构造函数 (Functions) 函数构建过程依赖于合约接口的 4bytes 函数选择器 (function selectors) 提取机制。系统首先扫描合约代码中所有有效的函数选择器，并解析其指向的跳转目标地址。每一个选择器的跳转目标块被标注为 **FUNCENTRY**，即函数入口点。从该入口块出发，递归遍历其可达的所有基本块，构建函数的完整子图。此外，还需特殊处理 **fallback** 函数逻辑——即当选择器未命中任何已知函数时的默认执行路径，确保所有潜在执行路径均被收录。至此，控制流图构建过程完成，整体设计如图3.3所示。系统将该 CFG 结构保存在内部数据库中，并支持在用户前端可视化展示，帮助用户全面理解合约字节码的控制逻辑与结构布局，为后续的漏洞模式识别和路径探索奠定坚实基础。

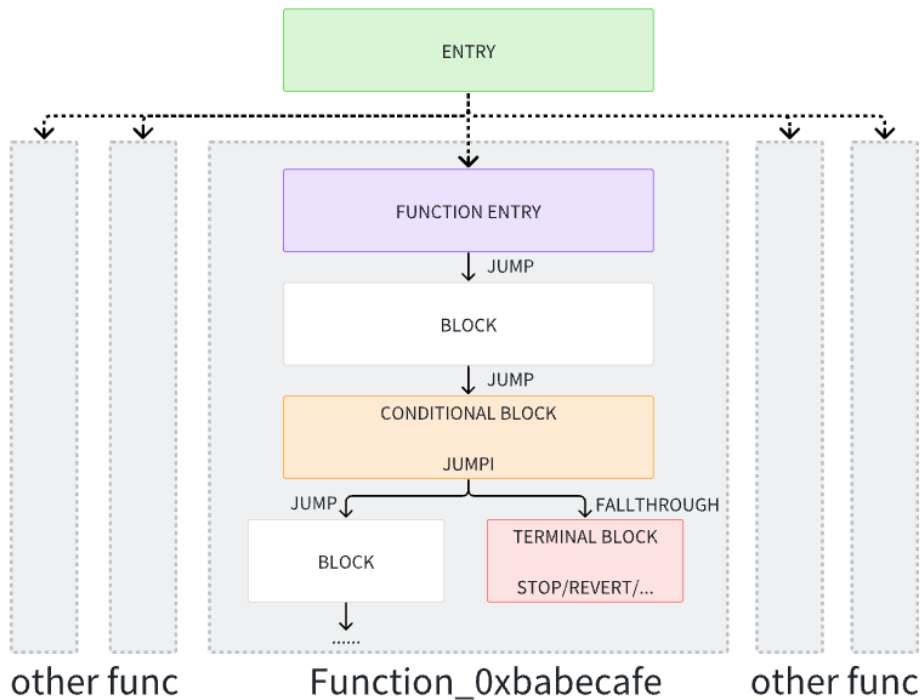


图 3.3 CFG 设计图

3.2.3 程序样本切片

在分析过程中，我们引入了若干关键符号以简化表达与建模：其中， S_b 表示反向切片 (Backward Slicing)，用于追踪变量或栈元素的来源； S_f 表示正向切片 (Forward Slicing)，用于分析变量在后续执行中的传播路径； $\mathcal{T}$ 则表示当前分析中的污点列表 (Taint Set)，记录被污染或具有潜在攻击价值的数据项。这些符号在后续的信息流分析与跳转修复中起到了重要的理论支持作用。

1. 初始化切片 反向切片的初始化过程首先确定目标指令执行前的栈状态。具体而言，系统将 `stacksize` 设置为目标指令所需的输入参数数量，这代表了执行该指令前栈必须达到的最小深度。同时，系统初始化一个空的切片结果列表 `backward_slice`，用于存储最终的指令序列。污染映射 `stack_taintmap` 被初始化为包含目标指令所有输入位置的集合，这些位置标记为“被污染”，表示需要追踪这些栈位置的数据来源。此外，系统还会获取目标指令所在基本块中位于该指令之前的所有指令，作为待分析的指令序列。栈变化量 `stack_delta` 和栈下溢标记 `stack_underflow` 都被初始化为 0，分别用于跟踪栈大小的变化和检测栈下溢情况。

2. 逆向遍历切片 逆向遍历过程采用优先队列机制从目标指令开始，按照控制流图的逆向路径分析每条指令对栈状态的影响。系统通过 `traverse_back` 函数实现基于优先级的搜索策略，确保重要的执行路径优先被探索。在遍历过程中，系统维护一个状态缓存机制，通过 `SlicingState` 类的哈希函数和等价性检查来避免重复计算相同的状态组合。系统逆序遍历指令序列，对每条指令首先判断其是否影响当前被污染的栈区域。如果指令的输出范围与被污染区域有重叠，则该指令成为切片候选。对于不同类型的指令，系统采用不同的污染传播策略，首先是普通指令：

$$\begin{aligned} & \{ins_1, ins_2, \dots, ins_k\} \cap \mathcal{T} \neq \emptyset \rightarrow \\ & \mathcal{T} := \{\mathcal{T} / \{out_1, out_2, \dots, out_m\} \cup \{ins_1, ins_2, \dots, ins_k\}\} \\ & S_b[ins] := \{ins\}, \text{ if } ins \text{ affect } \mathcal{T} \end{aligned}$$

对于 `DUP` 指令，如果栈顶位置被污染，系统会将污染传播到被复制的源位置：

$$\begin{aligned} & DUP[n] : \text{ if } stack_{top} \in \mathcal{T} \rightarrow \mathcal{T} := ((\mathcal{T} / stack_{top}) \cup stack_{top-n}) \\ & S_b[DUP[n]] := \{DUP[n]\}, \text{ if } stack_{top} \in \mathcal{T} \end{aligned}$$

对于 `SWAP` 指令，系统会交换两个位置的污染状态；对于普通指令，系统首先净化该指令写入的输出位置，然后污染该指令读取的输入位置：

$$\begin{aligned} & SWAP[n] : \text{ if } (stack_{top} \in \mathcal{T} \vee stack_{top-n} \in \mathcal{T}) \rightarrow \\ & \mathcal{T} := \text{ swap taint status between } stack_{top} \text{ and } stack_{top-n} \\ & S_b[SWAP[n]] := \{SWAP[n]\}, \text{ if } stack_{top} \text{ or } stack_{top-n} \in \mathcal{T} \end{aligned}$$

每当确定一条指令需要加入切片时，系统会调用栈对齐机制确保栈状态的一致性，并将该指令添加到切片结果中。为了减少冗余，系统在最终返回结果前会使用集合操作去除重复的切片序列，通过 `set(slices)` 将所有切片转换为元组后去重，再转换回列表

格式返回唯一的切片结果。整个过程持续进行，直到所有被污染的栈位置都找到了对应的数据源，或者达到了搜索边界。

3. 正向切片 除了反向切片，我们还在部分分析任务中采用了正向切片（Forward Slicing）技术，其用于评估某一污染输入在执行流程中所引发的后续影响路径。其实现原理与反向切片相对对称，因篇幅限制，此处不再赘述。

3.2.4 可达性分析器

基于污点传播和样本切片的静态分析器通常会通过扩大搜索范围来降低漏报率，但这一策略也不可避免地提高了误报率。为降低框架的误报率，本文实现了一个可达性分析器：该组件先对特定切片进行反向搜索与前向匹配，生成可能的执行路径；随后，利用下文介绍的模拟执行器对路径进行执行测试，以验证其是否真实可达，并将可达路径进行保存。在可达性分析过程中，我们将同类型漏洞的同一路径视为同一漏洞，从而有效减少了重复报告，显著降低了框架的整体误报率。实验结果详见 RQ5。

3.2.5 模拟执行器

为提升框架的可扩展性与分析能力，本文实现了一个面向 EVM 字节码的轻量级模拟执行器（Simulator），该组件虽然尚未完全符号化，但已具备基础的执行语义建模能力。模拟器内部构建了三大核心抽象：线性表模拟的 EVM 栈（Stack）、基于哈希表实现的内存模型（Memory）以及存储空间模型（Storage），可准确还原大多数智能合约在执行过程中的状态变更。通过该结构，框架能够执行局部指令序列，从而提供对字节码更具语义层级的理解。

在指令语义建模方面，我们参考 Solidity-Yul 官方开发手册的定义，为 EVM 字节码指令集中的主要算术类、环境访问类、控制流类及存储类指令分别实现了模拟语义。这种半符号化的执行方式在不依赖真实 EVM 环境的前提下，可有效捕捉执行路径、环境依赖以及潜在的数据依赖行为，为后续的污点分析、漏洞验证等任务奠定基础。为了验证模拟器的实用性，我们将 DoS 漏洞检测器生成的部分可疑漏洞路径作为输入，交由模拟器执行。实验结果见 RQ5

此外，我们可以通过分析控制流图（CFG）中函数入口点邻近指令的特征，能够在无 ABI 的条件下推断函数所需的参数数量与堆栈占用大小，进而实现参数填充与函数调用还原。这种方法虽无法还原类型信息，但可通过随机生成策略完成参数构造，为后续的模糊测试（fuzzing）提供支持。而无 ABI fuzzing 技术在现有字节码分析工具中尚属空白，具有一定的技术前沿性。

3.3 用户交互层

3.3.1 系统架构与核心功能总览

项目整体架构采用前后端分离设计，前端负责用户交互和可视化展示，后端基于高性能异步框架提供智能合约上传、漏洞分析与结果存储等核心服务。时序图3.4直观展示了客户端与服务器间的交互流程，从合约上传到漏洞检测及报告反馈，形成闭环审计体验。

技术栈与工具概述

前端部分基于现代 React 生态，使用 Next.js 构建高效的服务器端渲染页面，结合 Tailwind CSS 和 shadcn/ui 实现响应式的 UI 设计。核心图表与可视化依赖 D3.js 和 Chart.js，提供动态控制流图和审计报告图表展示。数据交互采用 RESTful API，异步请求由 axios 管理，确保流畅的用户体验。后端核心框架为 FastAPI，具备异步高性能和自动文档生成能力。uvicorn 提供高并发 ASGI 服务，结合 CORS Middleware 实现跨域资源安全访问。项目模块化结构清晰，包括路由分发、业务逻辑、数据模型及入口配置。

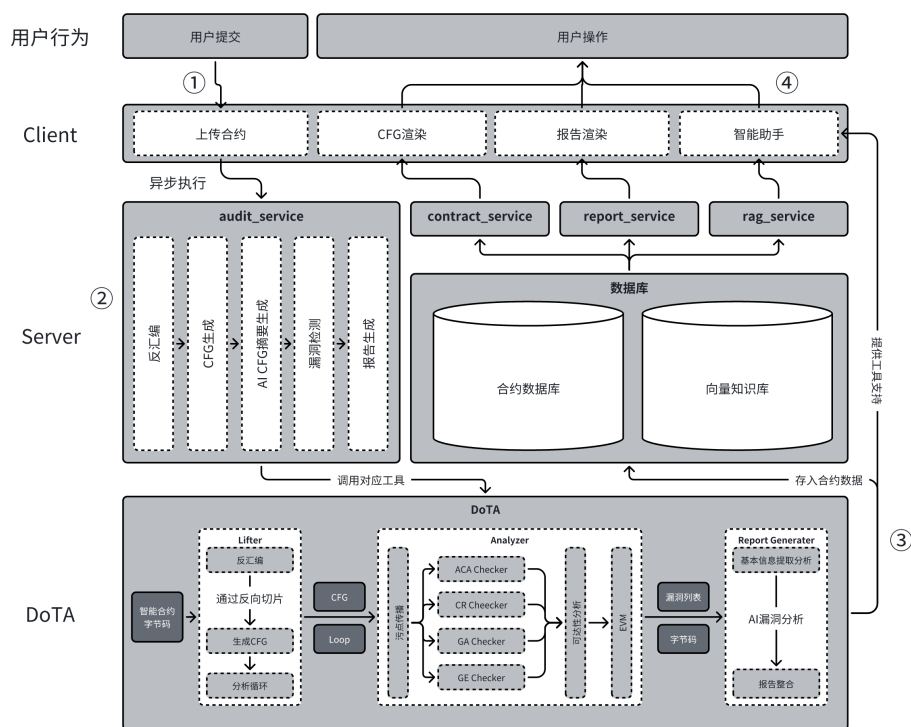


图 3.4 用户交互时序图

六大核心模块概述

1. 合约上传模块

该模块为用户提供多样化且灵活的合约导入方式，支持分析本地上传的 Solidity 源码文件、编译生成的字节码文件以及输入地址对应的部署在区块链上的合约。系统内置多层格式校验和安全验证机制，确保上传内容的合法性与完整性。前端通过异步通信实时反馈上传状态，保障用户操作流畅和透明。此模块是整个审计流程的起点，为后续分析奠定数据基础。

2. 漏洞检测模块

基于灵活的插件化检测架构，支持多种智能合约漏洞类型的自动识别。检测器可按需加载与扩展，适配不断演进的安全威胁。该模块不仅实现静态代码审计，还结合字节码层级的控制流分析，确保高准确率与低误报率，为合约安全保驾护航。

3. 控制流图 (CFG) 展示与交互模块

作为本系统的可视化核心，基于 D3.js 实现高交互性的控制流图渲染，直观展示合约内各基本块间的跳转关系及执行路径。通过自定义的圆角曲线连接块，配合搜索与缩略导航功能，用户能够快速定位关键信息。模块支持前驱后继节点快速跳转和路径跟踪，极大提升了用户对复杂合约逻辑的理解和审计效率。

4. 审计报告生成与历史管理模块

该模块自动整理检测结果，生成详尽且结构化的审计报告。报告包括漏洞分类统计、影响范围分析及具体修复建议，配合饼图、柱状图等可视化组件，帮助用户全面掌握合约安全状况。历史管理功能支持多维度检索与筛选，用户可快速访问过往审计记录，实现工作断点续查和项目追溯，方便团队协作与版本管理。

5. AI 智能审计助手模块

集成 AI 智能体 (Agent) 与 RAG (Retrieval-Augmented Generation) 技术，基于大语言模型构建自然语言交互界面。该助手可解答用户关于漏洞成因、审计细节及修复方案的各类问题，结合上下文与知识库进行精准响应。前端具备对话记录管理、历史查询及快捷引用功能，极大增强了审计过程中的辅助决策能力和用户体验。

6. 系统文档与用户支持模块

作为全流程用户引导与技术支持平台，文档模块系统性覆盖合约上传规范、CFG 分析机制、报告字段详解、AI 助手使用示例及常见问题解答。内容模块化，便于按需查阅，确保用户能够快速上手并深入理解系统功能。配合交互式帮助和示例演示，进一步降低使用门槛，提高平台的可用性和用户满意度。

3.3.2 CFG 分析与模块化漏洞检测

系统概览 为实现对智能合约的高效、可扩展的静态分析，我们构建了一个围绕控制流图（CFG）展开的审计系统架构。该系统主要由三个核心模块组成：CFG 分析模块、模块化漏洞检测框架以及结果可视化与报告生成模块。用户上传合约后，系统首先将字节码解析为指令序列，随后构建出完整的控制流图；图构建完成后，各类检测器将依次在图上运行，自动完成漏洞扫描；最终，分析结果以结构化列表、路径标注以及完整审计报告等多种形式展现，便于用户理解与溯源。这一架构设计的初衷是构建一个“可解释、可拓展、可交互”的智能合约分析平台。通过将字节码级分析与图结构相结合，系统能够捕捉复杂的程序跳转关系、条件路径和函数边界；而模块化检测框架则为漏洞分析逻辑提供了高度解耦的接口，使得不同专业背景的开发者的可以根据需求快速实现或替换特定漏洞检测器。这种整体设计极大地提高了系统的可维护性与可演化能力，为构建通用型、安全型的审计平台奠定了基础。



图 3.5 主界面

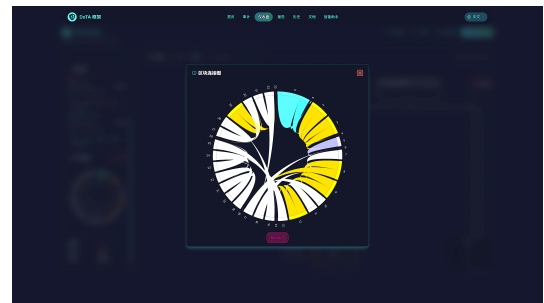


图 3.7 块间跳转和弦图

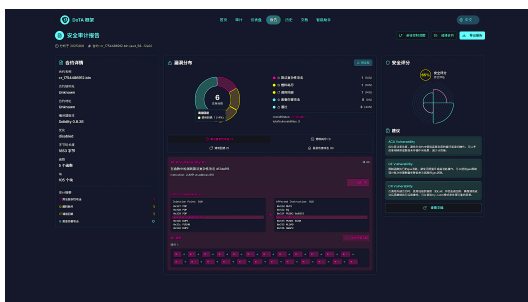


图 3.6 报告界面

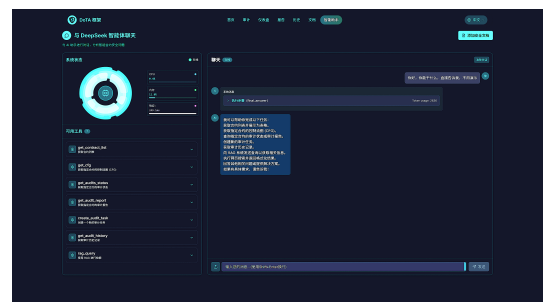


图 3.8 AI 智能助手

CFG 分析模块 控制流图（CFG）是本系统中承载字节码语义理解与漏洞检测逻辑的关键基础设施。系统在获取合约字节码后，会对其进行指令级解析，识别所有的 JUMPDEST 指令作为潜在的基本块入口，并根据 JUMP 与 JUMPI 等跳转指令构建图

的边结构，形成包含跳转路径与条件分支的有向图。每个节点表示一个基本块，其内部为连续的执行指令片段，边则表示可能的控制流路径。

a. 交互审计体验 在 CFG 构建完成后，系统为用户提供了丰富的图形交互功能，帮助其从不同粒度理解控制结构、精确定位关键路径并快速完成漏洞分析。整体交互设计以“可操作、可理解、可追溯”为核心原则，如图3.9所示，具体包括以下方面：

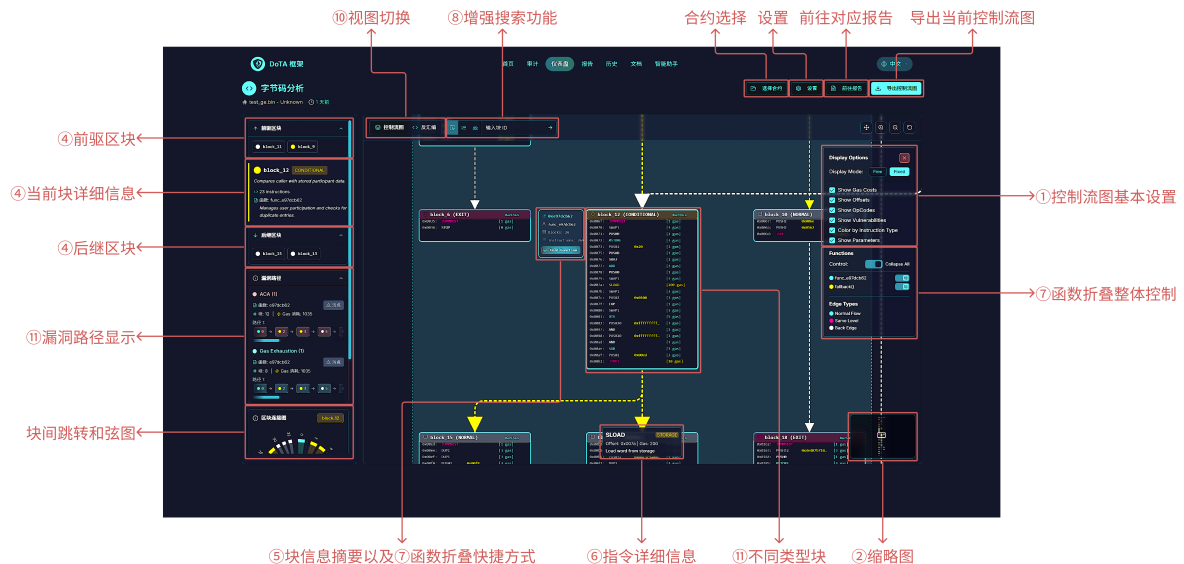


图 3.9 控制流图交互

1. **控制流图基础设置**：支持节点自动布局、图结构自动拉直、连接模式自定义等参数设置，保证图的结构清晰与可读性。
2. **缩放与缩略图导航**：提供平滑的图形缩放、平移操作，配合右下角缩略图导航网格以及键盘快捷键操作，便于在大型 CFG 中快速跳转与定位。
3. **节点高亮与路径追踪**：点击任意基本块后，会自动居中这个块并高亮其前后驱路劲，支持多层跳转链可视化，便于追踪执行流。
4. **前后继跳转控制**：用户可通过跳转按钮快速展开某个块的前驱块与后继块，并支持按调用深度分层控制跳转路径展开范围。同时当前块的详细系统中包含了 AI 生成的函数摘要与块摘要，辅助审计人员分析
5. **块信息摘要与详情**：每个基本块在悬浮时提供语义摘要（如所属函数、块大小、指令数量）。
6. **指令详情展示**：块内指令可在弹出窗口中查看，包括操作码、参数、偏移量等字段，并支持语义注释辅助阅读。

7. **函数折叠快捷方式**: 每个函数入口块附带快捷控制按钮, 点击即可折叠/展开该函数内部所有基本块, 便于聚焦主流程。
8. **搜索与跳转**: 用户可通过指令地址、函数名、语义关键词进行搜索并跳转定位至目标节点, 极大提升定位效率。
9. **点到即分析**: 支持点击任意节点后自动触发上下文分析, 调用路径、相关漏洞、上下文函数链一并显示, 简化分析流程。
10. **反汇编视图查看**: 支持仅反汇编视图代码的视图查看, 为对照分析提供了便利
11. **漏洞路径查看**: 显示检测到的漏洞执行路径以及对应污点, 为审计人员提供便利

b. 漏洞审计基础 CFG 分析为漏洞审计构建了坚实的语义基础, 系统中的各类漏洞检测器均以控制流图为核心输入, 从跳转关系、调用结构与数据依赖角度入手进行静态推理。每个基本块在构图阶段已被附带位置、跳转条件、执行边界等元信息, 便于检测器在图上进行图遍历、反向切片与污点传播等操作。更进一步, 系统支持对路径图进行指令级标注与编号, 结合 CFG 图中的高亮展示, 使得漏洞结果不仅是抽象的提示信息, 更可具体定位至某个路径段、某条边、甚至某条汇编指令。这种路径化的分析结果有助于提升用户对漏洞成因与攻击条件的理解, 也为后续修复与防御提供清晰依据。同时, 由于分析流程高度依赖图结构, 系统在性能设计上对图遍历与缓存做了专门优化, 确保大型合约在图构建与审计阶段的响应速度与可用性。

模块化漏洞测试框架 为了适应快速演化的智能合约安全需求, 我们设计并实现了一套模块化漏洞检测框架。该框架采用模块化检测器设计思想, 将每类漏洞检测逻辑封装为独立的插件, 每个检测器遵循统一的接口协议, 具有清晰的生命周期定义, 包括初始化、配置加载、执行入口、结果输出等阶段。用户或开发者只需将所需插件挂载至 Analyzer 组件, 即可在不更改主程序结构的前提下, 实现对新漏洞类型的检测支持, 真正做到“即插即用”。框架底层暴露了一系列稳定的分析接口, 包括基本块抽象语义访问、函数调用图、变量追踪、反向切片、路径重建、符号执行引擎包装器等, 检测器开发者可通过这些接口快速实现从模式匹配到符号分析等多种分析方式。以反向切片为例, 开发者仅需调用提供的 API 并传入目标指令地址与目标寄存器/变量, 即可返回控制依赖与数据依赖路径, 为检测触发条件提供充分上下文支持。

3.3.3 LLM 增强引擎

LLM 提供验证与修复建议 本项目集成国产大语言模型 DeepSeek, 以实现对智能合约漏洞的自动化验证与修复建议生成。在静态分析流程完成后, 系统将自动识别关键

漏洞元信息（如类型、位置、影响路径等），并构造标准化提示，提交至 LLM 进行分析。模型基于语义理解能力，生成具有上下文一致性的漏洞解释与修复建议，覆盖多类常见缺陷如重入、拒绝服务、Gas 耗尽等。该模块为审计人员提供了具有可读性与参考价值的审计结论，大幅提升了分析效率与专业性。

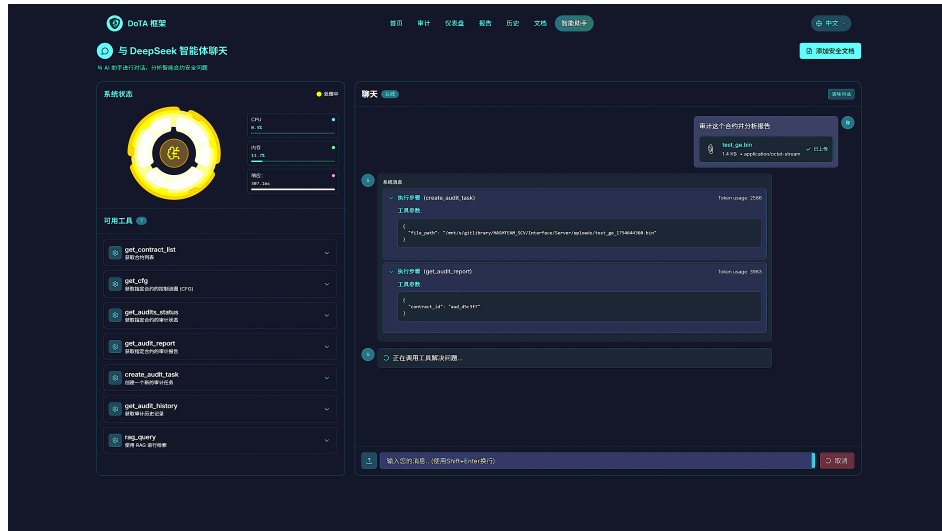


图 3.10 Agent 交互演示

RAG 检索增强生成机制 为进一步增强建议的准确性与可信性，系统引入 RAG 机制，使用国产向量化模型 BGE，为大语言模型提供可控、可信的知识背景。具体流程为：系统提取漏洞上下文构造查询向量，通过语义检索接口在本地构建的安全知识库中匹配相关内容，包括历史审计报告、漏洞利用模式、EIP 安全标准等文档。检索结果将作为模型输入的上下文补充，辅助生成更具事实依据、可解释性更强的响应内容，从而有效避免模型“幻觉”并增强分析稳定性。

基于 CodeAct 模式的漏洞分析 AI 智能体 我们基于 CodeAct [27] 模式构建了一个可以自动化漏洞分析的 AI 智能体，基座模型选用国产开源大语言模型 DeepSeek v3 671B，智能体采用基于代码执行的行动（Acting）模式，旨在提升智能体在复杂任务中的推理与执行能力。在面对复杂任务时，不仅依靠自然语言描述或简单指令，而是动态地生成可执行的代码片段，并在安全沙箱环境中运行，实现复杂逻辑处理和功能调用，从而实现更精细和灵活的任务完成。不仅可以利用 DoTA 平台丰富的外部调用接口，实现多样化功能，而且可以实现高度灵活的任务处理。

第四章 合约 DoS 分类与检测挑战

漏洞复现靶场

在对拒绝服务漏洞开展系统性研究的过程中，我们为了进一步验证此类漏洞在现有智能合约安全研究中相较于传统漏洞（如重入、权限控制失误等）所受到的关注程度较低，特地对 76 类智能合约常见漏洞类型进行了深入调研与分析。针对每一种漏洞类型，我们均编写了详细的漏洞复现说明文档，并构建了对应的靶场，以实现漏洞的自动化部署与复现。该靶场不仅涵盖了各类已被广泛研究的经典漏洞，还包含了部分较为边缘或新兴的攻击类型，具有良好的代表性与系统性。所有环境均在标准化基础上进行构建，便于今后的安全分析与自动化测试。

为促进社区交流与后续研究工作的开展，相关数据集将在本研究所参与的竞赛结束后正式开源至 GitHub 平台，供学术界与工业界共享与使用。

符号	符号含义	符号	符号含义
$\mathcal{V}$	漏洞集合	$\mathcal{L}$	循环集合
$\mathcal{W}$	工作列表	$\mathcal{E}$	外部跳转集合
$\mathcal{S}$	切片集合	$\mathcal{R}_0$	可达REVERT 的块

表 4.1 伪代码符号表

分类科学性与完备性说明：本文提出的四类智能合约拒绝服务（DoS）漏洞，即调用回滚攻击、算法复杂性攻击、GAS 耗尽和悲伤攻击，从攻击面、资源利用模式和逻辑影响三大核心维度出发，构成了对合约层面 DoS 威胁的完整描述。在攻击面上，调用回滚攻击针对批量外部调用与异常处理机制，算法复杂性攻击侧重恶意触发计算复杂度激增，GAS 耗尽聚焦对链上存储与遍历操作的过度消耗，而悲伤攻击则专门影响合约状态与时间依赖逻辑；从资源利用模式来看，这四类攻击分别对应外部调用路径控制、计算开销放大、Gas 配额榨取与状态锁定等截然不同的资源浪费策略；从逻辑影响角度而言，这一体系完整覆盖了可导致交易整体回退、计算执行中止、Gas 资源耗尽以及逻辑前提长期不成立等所有可能的服务拒绝场景，能全面刻画智能合约中可能导致 DoS 的风险空间。因此，这四类攻击类型彼此互补又相互独立，涵盖范围全面、分类边界清晰，既为理论研究提供了科学严谨的框架，也为自动化检测与防护手段设计奠定了完备的攻击面参考。

4.1 调用回滚攻击 (CR)

漏洞描述：调用回滚攻击 (Call Revert Attack, CR) 是一种智能合约中的拒绝服务 (DoS) 攻击形式。攻击者通过部署恶意合约，在接收以太坊转账时故意触发交易失败 (revert)，导致包含多笔转账的批量操作完全回滚。这种攻击的核心风险在于：当批量操作中的单次失败会导致整个交易状态回滚时，攻击者可利用此机制阻碍其他合法用户获取资金或服务。

样例代码片段：

```
1 contract DoSCallVictim {
2     address[] public sponsors;
3     function giveback() public onlyOwner {
4         for (uint256 i = 0; i < sponsors.length; i++) {
5             (bool _success, ) = sponsors[i].call{value: 1 ether}("");
6             require(_success, "transfer failed"); //单笔失败会使整个交易回滚
7         }
8     }
9 }
```

样例代码分析：该退款函数在循环中直接执行批量转账并检查结果，构成了典型的 DoS 攻击界面。其脆弱性在于：当攻击者作为接收方加入 `sponsors` 列表后，会在处理其转账时通过回调函数主动 revert。由于 `require(success)` 要求每笔交易必须成功，单次失败将导致整个交易链式回滚。攻击者通过精确定位这个单点故障，即可阻断所有后续转账操作，使得整个退款功能完全瘫痪。

检测挑战：识别此类攻击的主要障碍在于合约逻辑层面的隐蔽性：尽管检查转账结果的 `require` 语句在单次操作中属于安全实践，但在批次处理中却形成了 DoS 脆弱点。静态分析工具难以区分外部调用的恶意行为意图，也无法准确评估单点故障对整个系统的影响。检测需要深度理解转账依赖关系和失败处理机制，这在自动化分析中构成显著挑战。针对上述挑战，DoTA 依托高精度控制流图实现对程序逻辑的深层解析，通过细粒度 CFG 构建精准定位循环内的关键外部调用点及关联的失败处理逻辑，实现了精准的漏洞检测。

CR 检测 实现思路：首先，在合约的循环中查找满足以下条件的 CALL 指令：1) 该 CALL 指令转移以太币；2) 该 CALL 指令的目标地址受外部输入控制。这些 CALL 指令被标记为“可疑调用”。其次，计算出 CFG 中所有可以到达 REVERT 指令的基本块集合。最后，针对每一个“可疑调用”，检查其执行结果是否被一个后续的 JUMPI 指令直接用作跳转条件。如果当 CALL 执行失败时，该 JUMPI 指令的相应分支（跳转目标

或顺序执行路径) 能够到达前面计算出的可达 REVERT 的基本块集合中的任一基本块, 则认为存在 Call Revert 漏洞。

Algorithm 1: Call Revert (CR) 漏洞检测

Input: 控制流图 CFG **Output:** 漏洞集合 $\mathcal{V}$

```
1  $\mathcal{V} \leftarrow \emptyset, \mathcal{L} \leftarrow \text{Loops}(CFG);$ 
2 if  $\mathcal{L} = \emptyset$  then
3   return  $\mathcal{V}$ 

4 阶段一: 识别可疑 CALL 指令;
5  $\mathcal{S} \leftarrow \bigcup_{\substack{L \in \mathcal{L}, b \in L, i \in b \\ i = \text{CALL}, \text{Ether}(i)}} \{i \mid \text{backward}(i, \text{addr}) \cap \mathcal{E} \neq \emptyset\};$ 
6 其中  $\mathcal{E} = \{\text{CALLDATALOAD}, \text{SLOAD}, \dots\};$ 
7 if  $\mathcal{S} = \emptyset$  then
8   return  $\mathcal{V}$ 

9 阶段二: 标记可达 REVERT 的块;
10  $\mathcal{R}_0 \leftarrow \{b \mid \text{last}(b) = \text{REVERT}\}, \mathcal{R} \leftarrow \mu X. \mathcal{R}_0 \cup \{p \mid \exists b \in X, p \in \text{Pred}(b)\}$ 

11 阶段三: 验证跳转是否由 CALL 返回值控制;
12 foreach  $i \in \mathcal{S}$  do
13    $S_i \leftarrow \text{forward}(i, \text{ret});$ 
14   if  $\exists j \in S_i, j = \text{JUMPI}, \text{cond}(j) \triangleq i.\text{ret}, \text{fail}(j) \in \mathcal{R}$  then
15      $\mathcal{V} \leftarrow \mathcal{V} \cup \{(\text{func}(i), \text{id}(i), i)\};$ 

16 return  $\mathcal{V}$ 
```

4.2 算法复杂性攻击 (ACA)

漏洞描述：算法复杂性攻击 (Algorithm Complexity Attack, ACA) 是指攻击者通过精心构造输入数据，诱导智能合约在执行过程中触发计算复杂度较高的逻辑操作，从而导致大量消耗 Gas 资源，甚至耗尽调用者的 Gas 限额，造成合约调用失败。该类攻击的根本危害在于其可以造成链上资源浪费、服务阻断，严重时甚至形成拒绝服务 (DoS) 攻击。此类漏洞常见于依赖线性或更高复杂度的数据结构遍历、嵌套循环、排序、递归、无上限迭代等模式，若未对输入规模加以限制，极易被攻击者利用。

样例代码片段：

```
1 contract ExponentialLoop {
2     function exponentialLoop(uint n) public pure returns (uint) {
3         uint result = 0;
4         for (uint i = 0; i < n; i++) {
5             for (uint j = 0; j < n; j++) {
6                 for (uint k = 0; k < n; k++) {
7                     result += i + j + k;
8                 }
9             }
10        }
11        return result;
12    }
13 }
```

样例代码分析：该合约中的 `exponentialLoop` 函数包含三重嵌套的 `for` 循环，其时间复杂度为 $O(n^3)$ ，即输入参数 n 每增加一倍，函数的实际操作次数会以立方增长。由于该函数为 `pure` 类型，不涉及任何存储写入操作，因此在表面上看似安全、无副作用，但实际上，嵌套循环会对计算资源造成极大的压力。攻击者可以通过传入一个足够大的 n 值（例如几百），诱导函数执行数以百万计的迭代操作，从而迅速消耗掉调用者或交易的 Gas 限额。虽然函数最终返回一个确定性结果，但若在合约中将该函数嵌入更关键的逻辑路径（如合约部署、用户验证、状态更新前的校验），则会引发整个合约流程的失败甚至阻塞。此外，在极端情况下，这种设计还可能成为链上服务的瓶颈，给高频交互合约带来 DoS 威胁。该示例揭示了一个关键问题：即使没有资金相关操作，纯计算型函数也可能隐藏严重的资源滥用风险。

检测挑战：此类攻击的识别具有一定挑战性。首先，代码逻辑往往看起来完全合法、清晰、无副作用，不易在语义层面暴露出安全隐患；其次，传统静态分析工具主要聚焦于访问控制、重入、溢出等结构性漏洞，对于函数的输入规模与 Gas 消耗之间

的非线性关系难以精准建模与预警。DoTA 通过在更精确的控制流图上识别反向边，从而检测到提升算法复杂度的潜在风险，通过对该风险进行可行性分析，进而精确、有效地识别出算法复杂性漏洞。

ACA 检测 实现思路：首先获取所有合约中的循环，对这些循环的外部跳转块的 JUMPI 需要的栈上第一个参数——即影响“是否跳转”的控制变量——做反向切片，获取影响控制变量的根源变量，如果这些根源变量是否为外部变量（如 CALLDAT-ALOAD, CALLER），则证明用户的输入可以影响这些循环，也就是说算法的时间复杂度由外部输入决定，即算法复杂性漏洞。

Algorithm 2: 算法复杂性攻击检测流程

Input: 控制流图 CFG

Output: 漏洞集合 $\mathcal{V}$

```
1  $\mathcal{V} \leftarrow \emptyset, \mathcal{W} \leftarrow \emptyset;$ 
2 阶段一：识别外跳转块;
3 foreach  $L \in CFG.loops;$ 
4    $b \in L;$ 
5    $last(b) = JUMPI$  do
6     if  $b.fallthrough\_id \notin L$  or  $\exists t \in b.jump\_target\_id : t \notin L$  then
7        $\mathcal{W} \leftarrow \mathcal{W} \cup \{b\};$ 
8 阶段二：追踪外部依赖跳转条件;
9 foreach  $b \in \mathcal{W}$  do
10   foreach  $s \in backwardSlice(last(b), taintmap)$  do
11     if  $\exists i \in s : i.name \in EXTERNAL\_OPCODES$  then
12        $\mathcal{V} \leftarrow \mathcal{V} \cup (func(b), id(b), JUMPI, i);$  break;
13 return  $\mathcal{V};$ 
```

4.3 GAS 耗尽 (GE)

漏洞描述：GAS 耗尽 (Gas Exhaustion s.t. GE) 是指在合约中对 Gas 密集型操作的误用，攻击者甚至不需要经过构造数据就能通过合约来使合约瘫痪。合约中多次调用了会消耗大量 Gas 的操作，如在循环中操作一个全局列表。此类攻击常见于遍历、排序、嵌套循环等场景中。

样例代码片段：

```
1 contract GE {
2     address[] entrants;
3     function enter() public {
4         for (uint256 i; i < entrants.length; i++) {
5             if (entrants[i] == msg.sender) {
6                 revert("You've already entered!");}}
7         entrants.push(msg.sender);}}
```

样例代码分析：该合约首先定义了一个全局的数组，这意味着这个数组会被存在 storage 中，然后在 enter 函数中合约通过循环检查是否有当前地址进入过合约，如果没有就将其写入列表。每次有用户 enter 时，合约都需要遍历整个数组，同时可能会有写入的操作，这样就会导致大量的 Gas 消耗，甚至瘫痪整个合约。

检测挑战：这类攻击的识别具有一定难度，因为这是一种逻辑层面的攻击，并不存在代码语义层面的问题，传统的静态分析工具难以从逻辑层面检测这种漏洞。有效检测该漏洞的方式依赖于对循环体中的高 gas 操作。DoTA 构建了更为精确的控制流图，极大地提高了循环体识别的准确率，因此提高了 GE 漏洞检测的效率与精度。

GE 检测 实现思路：首先获取所有合约中的循环，然后标记含有数组操作的循环。对这些循环的外部跳转块的 JUMPI 做反向切片，查看是否有 SLOAD 影响；若有，则继续对 SLOAD 做反向切片，查看读取的位置并记录下来。接着查看全局的 SSTORE，查看操作的位置是否和 SLOAD 的位置一样，若有，则记录为漏洞注入点。

Algorithm 3: Gas Exhaustion Attack 检测流程

Input: 控制流图 CFG **Output:** 漏洞集合 $\mathcal{V}$

```
1  $\mathcal{V} \leftarrow \emptyset;$ 
2  $Slots \leftarrow \emptyset;$ 

3 阶段一：定位可疑循环与跳出块;
4  $JumpBlocks \leftarrow \{b \mid L \in CFG.loops, \exists i \in L : i \in LIST\_OPCODE, b \in L, last(b) =$ 
    $JUMPI, fallthrough(b) \notin L \text{ or } \exists t \in targets(b) : t \notin L\};$ 

5 阶段二：追踪读槽指令;
6 foreach  $b \in JumpBlocks$  do
7   foreach  $i \in backwardSlice(last(b), \{0\})$  do
8     if  $i \in \{SLOAD, SLOADBYTES\}$  then
9        $Slots \leftarrow Slots \cup \{last(backwardSlice(i, \{0\}))\};$ 

10 阶段三：匹配 SSTORE 写入;
11 foreach  $w \in CFG.instructions, w = SSTORE$  do
12    $p \leftarrow last(backwardSlice(w, \{1\}));$ 
13   if  $p \in Slots$  and  $p.args \neq 0xdeadcode$  then
14      $\mathcal{V} \leftarrow \mathcal{V} \cup \langle \text{"Gas Exhaustion"}, func(p), id(p), p, w \rangle;$ 

15 return  $\mathcal{V};$ 
```

4.4 悲伤攻击 (GA)

漏洞描述：悲伤攻击 (Griefing Attacks s.t. GA) 是一种通常与业务逻辑有关的攻击。这些攻击对智能合约系统的运行造成了负面影响，尽管攻击者没有直接获利。在某些情况下，恶意攻击可能会被部署来破坏整体系统运行或在特定的关键时刻造成破坏。

一个典型场景为：在构造函数中，受益人被设置为部署者的地址以及自定义延迟，例如 24 小时。任何人都可以存入资金，这些资金在配置的延迟后可以提取并转移给 beneficiary。但是，每笔存款必须将非零数量的 ETH 转移到智能合约。攻击者可以通过在延迟期结束前提交交易来重置时间戳来形成这个攻击。或者，攻击者可以参与抢先交易、预测和抢占受益人对 withdraw 函数的调用，从而创建更具成本效益的拒绝服务攻击。

样例代码片段：

```
1 contract DelayedWithdrawal {
2     address beneficiary;
3     uint256 delay;
4     uint256 lastDeposit;
5
6     constructor(uint256 _delay) {
7         ...
8     }
9
10    modifier checkDelay() {
11        require(block.timestamp >= lastDeposit + delay, "Keep waiting");
12        _;
13    }
14
15    function deposit() public payable {
16        require(msg.value != 0);
17        lastDeposit = block.timestamp;
18    }
19
20    function withdraw() public checkDelay {
21        (bool success, ) = beneficiary.call{value: address(this).balance}("");
22        require(success, "Transfer failed");
23    }
24 }
```

样例代码分析：该合约实现了一个带延迟的提款机制：任何人都可以向 deposit()

函数转入 ETH，并将 lastDeposit 时间戳更新为当前区块时间；只有当前区块时间满足 $\text{block.timestamp} \geq \text{lastDeposit} + \text{delay}$ 时，beneficiary 才能调用 withdraw() 函数，通过低级调用一次性提取全部余额。表面上，这一设计旨在确保资金在最后一次存款之后静置一段时间才可被提取，避免 beneficiary 过早支取。然而，攻击者可以反复以极低面额（甚至 1 wei）调用 deposit()，持续把 lastDeposit 刷新为当前区块时间。只要攻击者在每个 delay 窗口到期前再次存款，便可无限期阻止 withdraw() 生效，导致 beneficiary 无法提取资金。

检测挑战：这类攻击是一种逻辑层面的 DoS，不依赖外部调用失败或回退，只利用合约自身的状态更新逻辑就能锁死提款路径。为了有效地检测此类漏洞，传统的思想是需要在执行路径探索的基础上引入基于模拟执行与模糊测试的机制，从而生成多样化的输入数据，观察合约在不同调用场景下资源消耗的变化，重点关注是否能够通过低成本构造高损失，而这样的检测策略的效率较低。相反，DoTA 通过在程序中寻找存储在 storage 中的核心变量，通过静态分析外部输入是否能够控制核心变量，进而确定该合约是否有被 griefing attack 的风险，大大提高了分析的效率。

GA 检测 实现思路：首先提取合约中存放在 storage 中的的关键变量，是否有外部变量可以污染这些关键变量，在该例中为向 storage 中存入 TIMESTAMP，如果有，标记为可疑合约。对于可疑合约，然后遍历所有可能到达 REVERT 的基本块，找到所有可能导致 Griefing Attack 的基本块，通过检测 TIMESTAMP 是否能够控制跳转来判断是否存在可能导致 Griefing Attack 的逻辑。

Algorithm 4: Griefing Attack 检测流程

Input: 控制流图 CFG **Output:** 漏洞集合 $\mathcal{V}$

```
1  $\mathcal{V} \leftarrow \emptyset$ ;  
2 阶段一: 筛选可疑合约;  
3 if 不存在  $SSTORE \leftarrow TIMESTAMP$  then  
4   return  $\mathcal{V}$ ; // 未发现依赖  $TIMESTAMP$  的存储操作  
5 阶段二: 反向传播  $REVERT$  可达块;  
6  $\mathcal{W} \leftarrow ReachRevert \leftarrow \{b \mid last(b) = REVERT\}$ ;  
7 while  $\mathcal{W} \neq \emptyset$  do  
8   foreach 前驱  $p$  of  $b \leftarrow pop(\mathcal{W})$  do  
9     if  $p \notin ReachRevert$  then  
10       $ReachRevert \cup \leftarrow \{p\}$ ,  $\mathcal{W} \leftarrow \mathcal{W} \cup \{p\}$ ;  
11 阶段三: 匹配  $JUMPI$  跳转条件;  
12 foreach  $b \in ReachRevert$ ,  $last(b) = JUMPI$  do  
13   if  $TIMESTAMP \in backwardSlice(last(b), \{0\})$  then  
14      $\mathcal{V} \leftarrow \mathcal{V} \cup (func(b), id(b), TIMESTAMP, JUMPI)$ ;  
15 return  $\mathcal{V}$ ;
```

第五章 实验与评估

为了评估 DoTA 的表现，我们通过回答以下几个 Research Questions 来测评我们的工具：

RQ1: DoTA 在检测智能合约漏洞方面具有怎样的有效性？

RQ2: 与当前主流工具相比，DoTA 在检测已知漏洞方面表现如何？

RQ3: DoTA 的特定组件（如 `hard_xref`）在提升 CFG 精度方面是否不可或缺？

RQ4: DoTA 在分析大规模合约时具有怎样的可扩展性？

RQ5: DoTA 的动态验证模块是否不可或缺，且能有效降低误报率？

我们在一台具有 12 核的 12th Gen Intel(R) Core(TM) i5-12500H CPU 上完成了我们的实验，机带 RAM 为 16GB，操作系统为 Ubuntu 22.04.5 LTS.

5.1 RQ1: DoTA 在检测智能合约漏洞方面具有怎样的有效性？

数据集构造 为全面评估我们漏洞检测工具的效果，我们系统性地收集了来自顶级学术会议论文和知名区块链安全公司发布的技术报告，整理并复现了四类典型的智能合约漏洞实例，分别为：算法复杂性攻击（ACA）、控制流异常漏洞（CR）、悲伤攻击（Griefing Attack, GA）和 Gas 耗尽漏洞（Gas Exhaustion, GE）。我们总计收录了 50 个真实漏洞合约，分别为 9 个 ACA 漏洞、20 个 CR 漏洞、10 个 GA 漏洞和 11 个 GE 漏洞，统称为集合 V 。为了更准确地评估工具的误报能力，我们基于上述漏洞合约构造了对应的无漏洞版本，记为 $\bar{V}$ ，通过修补相关逻辑使其不再触发原有的漏洞。例如，在 CR 漏洞中，我们修复了错误跳转逻辑；在 GA 漏洞中，我们移除了恶意报酬路径；在 GE 漏洞中，我们对无限循环或递归逻辑进行了优化或终止条件补充。最终，共生成了 31 个良性样本，用于负样本检测实验。我们基于上述数据集，在统一测试环境中对每一类漏洞进行检测，并评估了工具的精确率（Precision）、召回率（Recall）、准确率（Accuracy）以及平均分析时间（AvgT）。结果如表 5.1 所示：

对于 ACA（算法复杂性漏洞），我们的工具能够完全捕获所有漏洞（ $TP = 9$ ），由于 ACA 的良性样本通常是不使用这些算法，因此我们没有设计相关的良性样本，因而没有召回率和准确率的数据。分析平均耗时 116.25ms，具备较好的效率。CR（控制流异常）漏洞的检测具有一定挑战性。我们检测出 14 个真实漏洞（TP），但漏报 6 个（FN），造成召回率仅为 70%。不过由于所有负样本均未被误报（ $FP = 0$ ），精确率保持在 100%。准确率也同步为 70%。这说明我们在控制流异常场景下具备较强的保守性，但在复杂跳转关系的识别上仍有改进空间。GA（悲伤攻击）漏洞检测中，我

们成功识别了 8 个漏洞样本 ($TP = 8$)，全部召回 ($FN = 0$)，召回率达 100%。但存在 4 个良性样本被误报 ($FP = 4$)，导致精确率下降至 66.7%，准确率为 80%。这一现象说明，在处理复杂的共享状态修改与报酬边界检测方面，我们的静态分析策略可能存在一定过拟合。而 GE (Gas 耗尽) 漏洞的检测效果最为全面和可靠。该类漏洞普遍存在显式的递归或循环结构，工具准确地识别出了所有漏洞合约 ($TP = 11$) 和全部良性合约 ($TN = 10$)，精确率、召回率与准确率均为 100%。此外，该类漏洞的平均分析时间仅为 5.2ms，表明该检测逻辑在效率上具有极强优势，适合部署于大规模批量审计任务中。

VULN	V	$\bar{V}$	TP	FN	TN	FP	Prec.	Rec.	Acc.	AvgT.(ms)
ACA	9	—	9	0	—	—	—	—	100.0%	116.25
CR	20	10	14	6	10	0	100.0%	70.0%	70.0%	61.2
GA	10	10	8	0	6	4	66.7%	100.0%	80.0%	475.9
GE	11	11	11	0	10	0	100.0%	100.0%	100.0%	5.2

表 5.1 四种漏洞的检测性能表现

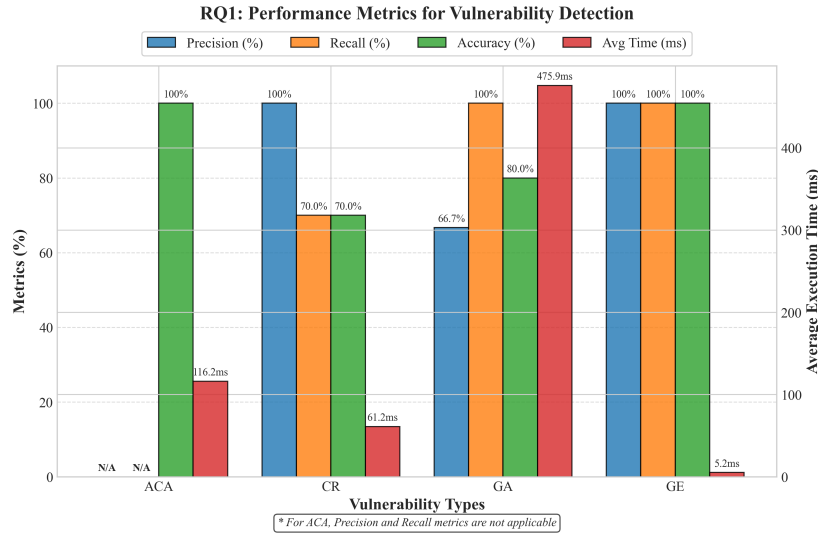


图 5.1 四种漏洞的检测性能表现

综上所述，我们的工具在四类漏洞中均表现出良好的静态分析能力，尤其在 ACA 和 GE 类漏洞上达到了准确性与效率的双优；对于结构更复杂的 CR 与 GA 漏洞，虽然在漏报与误报上仍有优化空间，但整体检测表现依然优于多数已有方案。未来，我们将进一步提升控制流建模的精度与语义分析的深度，以进一步提升检测工具的泛

化能力与实用性。

5.2 RQ2: 与当前主流工具相比，DoTA 在检测已知漏洞方面表现如何？

通过对现有智能合约安全检测工具的全面调研，我们发现当前学术界尚未提出能够有效识别算法复杂性漏洞和悲伤攻击的自动化解决方案。为客观评估 DoTA 框架的检测效能，本研究选取了两种具有代表性的基准工具进行对比分析：静态分析工具 Slither（基于源代码分析）和 ISSTA'22 会议提出的符号执行工具 eTainter，这两种工具均声称能对调用回滚漏洞进行检测。为确保实验的公平性，我们采用 RQ1 中构建的标准 CR 漏洞数据集进行系统性评估。

Tool	TP	FN	FP	TN	Prec.	Rec.	Acc.	F1 score
DoTA	14	6	0	10	100.00%	70.00%	80.00%	82.35%
slither	20	0	9	1	68.97%	100.00%	70.00%	81.63%
eTainter	0	20	10	10	0.00%	0.00%	33.33%	0.00%

表 5.2 不同工具检测 CR 漏洞的表现

表5.2展示了详细的对比实验结果。从检测性能来看，DoTA 展现出最佳的平衡性特征，在保持 100% 精确率（零误报）的同时实现了 70% 的召回率，F1 分数达到 82.35%。相比之下，Slither 虽然实现了 100% 的召回率（零漏报），但其高达 31% 的误报率（9/29）导致精确率降至 68.97%；而 eTainter 在当前测试集中完全未能识别任何 CR 漏洞，表现出明显的功能局限性。图5.2通过可视化形式进一步印证了这些发现。值得注意的是，Slither 的高召回率是以牺牲精确性为代价的——其对 10 个良性样本仅能正确识别 1 个，这种检测策略在实际部署场景中将产生严重的运维负担。相比之下，DoTA 采用的混合分析方法（结合控制流精化和上下文敏感的污点传播）在保证零误报的同时，仍能检测出 70% 的真实漏洞，这种平衡特性对于智能合约的实时安全监测具有重要实践价值，为区块链应用的规模化部署提供了可靠的安全保障。

5.3 RQ3: DoTA 的特定组件在提升 CFG 生成精度方面是否不可或缺？

Baselines. 尽管当前已有若干研究工具致力于检测拒绝服务 (DoS) 类漏洞，如 **eTainter** [16] 和 **DoSChecker** [17]，但这些工具在复现性与功能完整性方面仍存在诸多限制，因而无法直接用于全面对比分析。首先，一些工具未公开其源码，尤其是 DoSChecker 的实现细节无法获取，因此我们无法对其进行复现实验。为此，我们选择了在 CCF-A

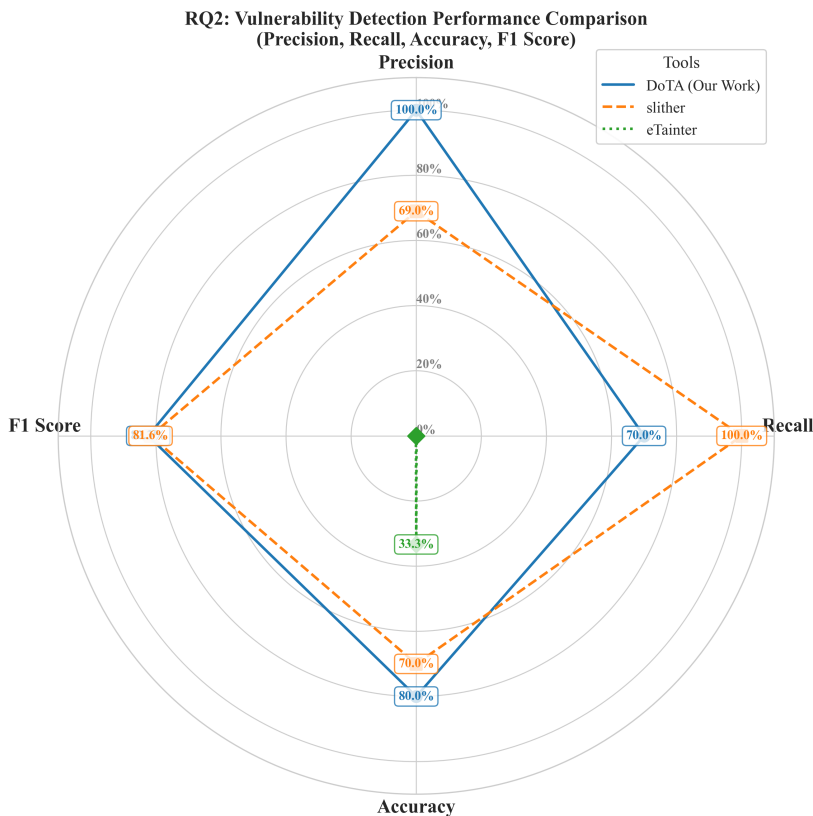


图 5.2 不同工具检测 CR 漏洞的表现

类会议（如 USENIX、IEEE S&P、ISSTA）中公开实现的控制流图（CFG）构造工具，如 AVVerifier，作为实验基准。这些工具具备可获取源码、逻辑可复查的特点，更适合用于系统性实验对比。其次，部分工具尽管提供了完整的 CFG 构造能力，但本身并未实现针对循环结构的检测逻辑。因此，为了保持公平性和一致性，我们为这些工具补充实现了与 DoTA 相同的循环检测模块，以便在相同的前提下评估其 CFG 构造精度对于漏洞检测能力的影响。此外，为了定量评估 DoTA 中关键模块 `hard_xref` 对 CFG 构造精度的影响，我们设计了对照实验，构造了一个移除 `hard_xref` 的 DoTA 版本（记作 DoTA_w/o_xref），并在相同的数据集上进行了循环检测实验。实验结果如表 5.3 所示。

从表 5.3 的实验结果可以看出，AVVerifier (USENIX'24) 在控制流图（CFG）构造过程中仅使用了 `simple_xref` 机制，该机制只能识别明确存在的直接跳转（如绝对跳转、`call` 指令等），而无法覆盖如间接跳转、延迟跳转、寄存器跳转等复杂边关系。这种策略虽然实现简单，但大大削弱了 CFG 的完整性与精确度，使得循环结构在控制流中容易被漏识，最终影响下游分析任务的准确性。我们构造的 DoTA_w/o_xref 版本也采用了与 AVVerifier 相同的跳转边策略，结果显示其在循环检测方面与 AVVerifier

Algorithm/Tool	Actual Loops	DoTA	DoTA_without_xref	AVVerifier	eTainter
BFS	3	2	0	0	5
BubbleSort	2	3	0	0	1
CountingSort	3	2	0	0	2
Fibonacci	1	1	0	0	0
LinearSearch	2	1	0	0	1
LCS	3	3	0	1	4
MatrixMul	3	4	1	1	2
PolyEvaluation	3	1	0	0	0
Power	2	1	0	0	0
QuickSort	3	5	2	0	3

表 5.3 不同工具在基准测试中检测到的循环

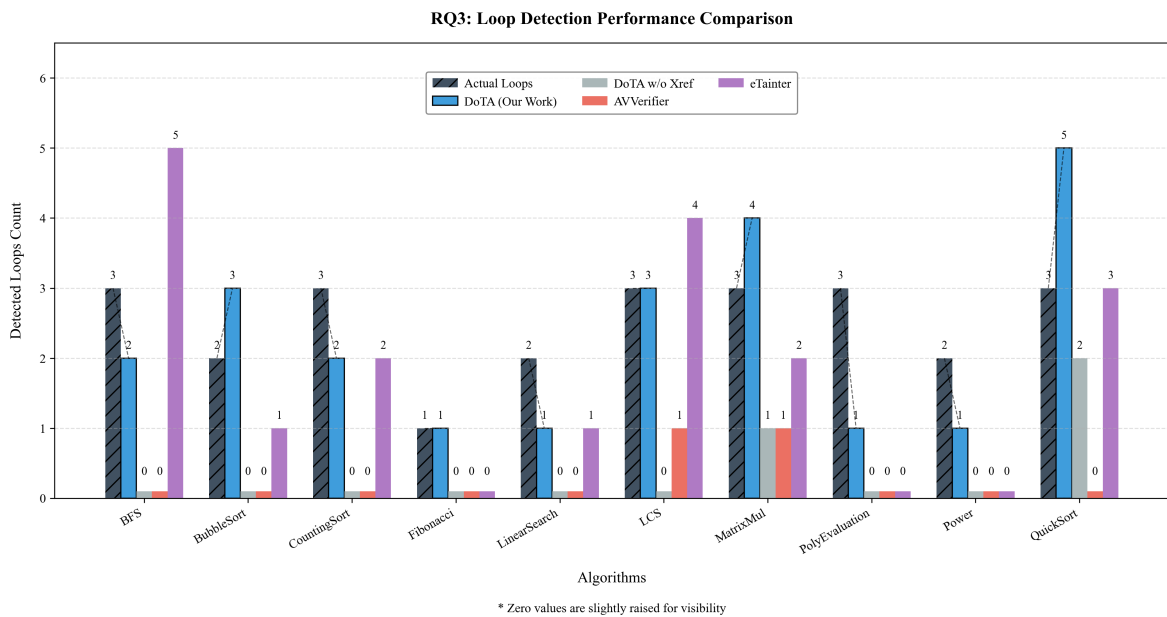


图 5.3 不同算法下各工具循环检测数量对比

表现一致，均存在较为严重的漏检现象。这一对照实验进一步验证了我们在 DoTA 中设计的 `hard_xref` 模块对于识别复杂跳转边的必要性与有效性，它显著提升了 CFG 构造的覆盖范围和精度。

更为关键的是，DoS 漏洞的本质通常源于程序中特定循环路径上的资源耗尽行为，因而循环结构的准确识别是漏洞检测的基础。一旦漏识循环节点，漏洞将难以被

捕获，造成严重的漏报（False Negatives）。在这一背景下，DoTA 中集成的困难边查找机制为 CFG 构建提供了更高的语义完整性，显著提升了系统对潜在 DoS 漏洞的检测能力与鲁棒性。

此外，虽然 eTainter 同样实现了类似的困难边查找功能，其在循环检测方面相较于 DoTA_w/o_xref 和 AVVerifier 表现更好，但由于其内部循环识别逻辑过于粗糙，导致其在多组样本中出现了较高的误报与漏报现象。例如在 BFS、LCS 等典型算法上，eTainter 无法准确区分循环层级与边界，反而对后续漏洞分析产生负面影响。

需要指出的是，DoTA 在某些场景下也可能存在一定的误报或漏报。例如在 BFS 样例中，系统仅检测出两个循环，而人工标注为三个。这是由于该程序中存在嵌套循环结构，两个子循环在控制流上表现为一个融合循环，这种合并在实际漏洞传播路径上不会带来影响，因此对最终漏洞分析结果无实质性干扰。又如在 BubbleSort 中，DoTA 额外识别出了一个由 if-else 结构产生的分支路径，该路径尽管并非严格意义上的“循环”，但在静态分析框架中有助于保守建模，提升污点传播路径的覆盖率，从而提高整体检测的准确性。

综上所述，本文提出的 DoTA 工具在 CFG 构造、复杂跳转边解析与循环检测方面展现出显著优势，不仅显著降低了现有工具在循环识别方面的误报与漏报问题，也为后续基于污点分析的 DoS 漏洞检测提供了更加稳健的基础。这一能力对于静态分析系统在工业级程序上的适应性和实用性具有重要推动作用。

5.4 RQ4: DoTA 在分析大规模合约时具有怎样的可扩展性？

数据集构建 为全面评估 DoTA 框架在大规模智能合约分析场景下的可扩展性，我们设计了一套系统性评估方案。实验数据来源于以太坊主网区块高度 22559856 至 22687017 区间内所有至少完成一次交易交互的智能合约。通过严格的去重处理和有效性验证，最终构建了包含 15,949 份真实合约的测试数据集，这些合约涵盖了金融、游戏、NFT 等多个主流应用场景，具有充分的行业代表性。

实验结果显示，DoTA 在当前配置下（详见本章实验环境说明）成功完成了 71.23% 合约的分析任务，检测到 342 份合约存在潜在 DoS 漏洞（占比 3.01%），在测试数据集中，有 4588 份合约（占比 28.77%）因分析超时或编译器版本过旧未能完成检测，其中 3961 份合约是因编译器版本问题导致失败，平均超时时间为 187 秒。

热力图（图 5.4）清晰揭示了合约部署呈现强烈的阶段性爆发特征，而非均匀分布，红橙色高密度区域集中出现在特定区块区间，这与以太坊网络升级或市场热点事件高度同步，凸显了生态事件对开发活动的驱动效应。柱状图（图 5.5）进一步量化

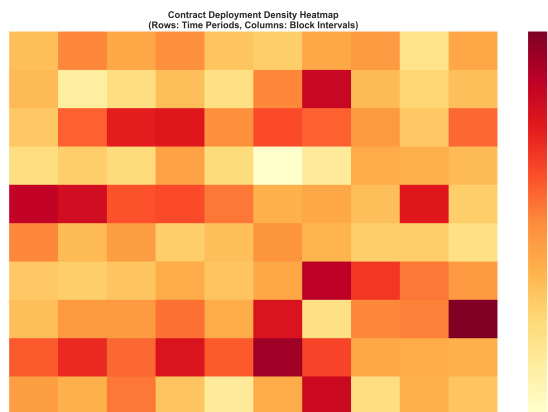


图 5.4 合约部署的区块区间与时间段的
热力图

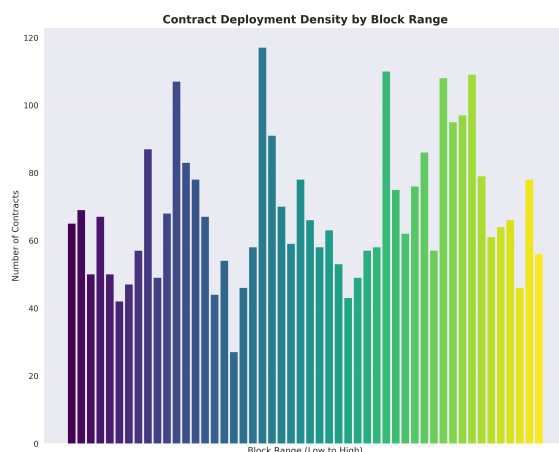


图 5.5 区块部署的合约数量柱状图

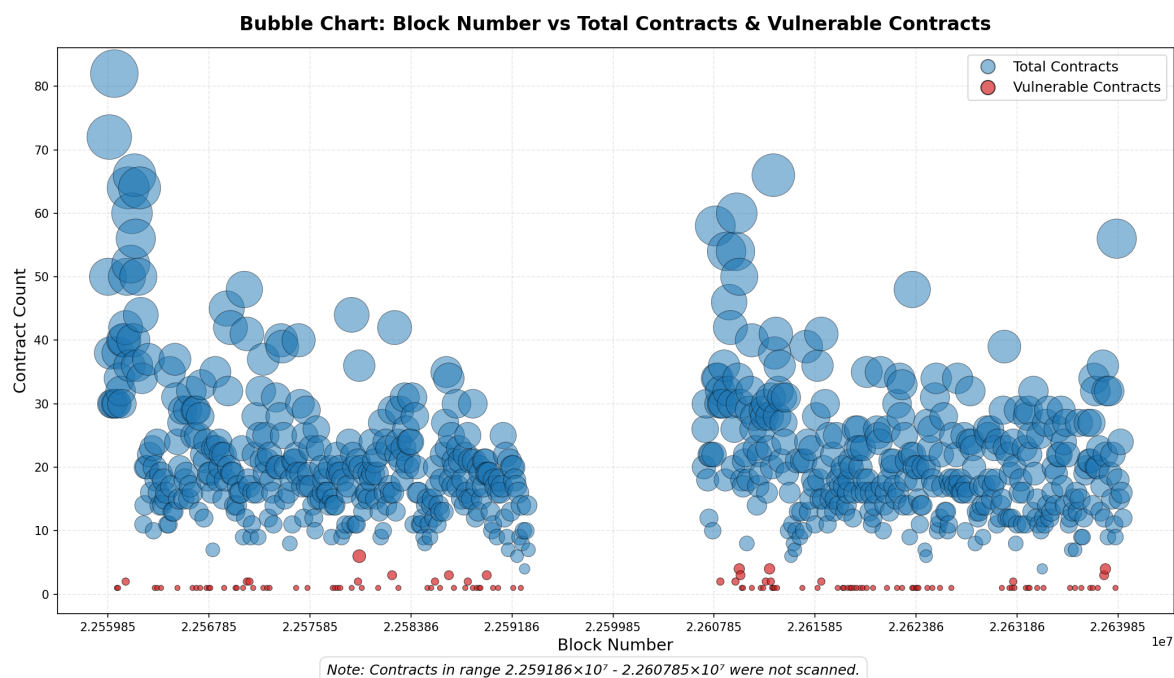


图 5.6 所有合约与漏洞合约和区块号的气泡图

了这种不均衡性——少数区块范围出现极端部署峰值。气泡图（图 5.6）则动态呈现了规模化分析的价值，对全部合约和存在漏洞和合约在不同区块的分布情况进行了可视化，展示了我们的工具在大规模数据集上的分析效果。

通过 profiling 分析发现，当前框架的主要性能瓶颈在于 `networkx` 库处理大型控制流图时的效率问题。具体表现为在分析具有复杂控制结构的合约时，图遍历算法时间复杂度呈非线性增长；内存消耗随基本块数量增加而快速上升。因此我们提出以下优化方案：开发面向字节码控制流图的算法，利用智能合约控制流特有的线性结构和跳转模式进行优化；实现增量式分析策略，对已分析过的子图进行缓存复用。这些优

化将作为我们未来工作的重点。值得注意的是，即便在当前版本下，DoTA 已展现出良好的工程适用性，完全满足对以太坊每日新增合约量的实时分析需求。

5.5 RQ5: DoTA 的动态验证模块是否不可或缺，且能有效降低误报率？

为验证动态验证模块的必要性与效果，我们选取了一些在静态分析中容易产生重复报告的合约及其变异版本，设计了消融实验，相关实验结果表 5.4 所示：

Contract Name	Actual Vulnerabilities	Static Only	Full Procedure
Enter List	2	4	2
Time Lock	1	2	1
String Operations	3	12	4
Hash	3	11	3
Call Revert	2	5	3
Two Loop	2	4	2
Audited Vault	8	10	9
Vault Filesystem	2	6	3
Deposit Tracker	2	8	2
Sorted Vault	4	10	6
Processed Vault	1	1	1
Whitelist Vault	3	3	3
Event Vault	3	9	4
Balance Vault	3	12	7
User Registry	7	15	10
Dictionary	8	8	8
Voting System	5	8	7
Inventory System	1	1	1
Reserve System	1	1	1
SC Manager	3	7	3

表 5.4 完整检测和仅静态检测检测到漏洞个数的差别

Baselines. 虽然我们的工具在漏洞检测的漏报率方面表现良好，但在误报率上仍存在一定不足。这主要是由于静态分析在理论上倾向于扩大搜索范围以减少漏报，结果

可能导致多个漏洞切片实际上源于同一执行路径。此外，静态工具仅依赖污点传播生成的切片，难以准确还原真实的执行流程，从而进一步增加误报风险。因此我们分别在静态分析阶段和动态验证阶段各记录一次漏洞数据，对比了前后结果。

从表 5.4 的实验结果可以看出，仅使用静态分析时，检测到的漏洞数量普遍高于真实漏洞数量，在部分合约中甚至出现了成倍的差异，例如 String Operations 合约中静态检测报告了 12 个漏洞，而实际仅有 3 个。这一现象验证了前文所述：静态分析为降低漏报率而扩大搜索范围，导致多个报告源自同一执行路径，从而引入了大量冗余结果。

在引入动态验证模块后（Full Procedure），漏洞检测结果与真实漏洞数量的差距显著缩小，部分合约如 Enter List、Time Lock、Deposit Tracker 等，误报被完全消除，检测结果与真实情况完全一致。在其他复杂合约中，虽然仍存在少量误报，但整体误报率较静态分析阶段显著降低。

因此，可以得出结论：动态验证模块在 DoTA 中并非可有可无，而是有效抑制静态分析固有的误报问题的重要环节。它不仅能够显著减少重复报告，还能更准确地还原实际执行路径，从而提升审计结果的可靠性和实用价值。

第六章 总结与讨论

本研究提出了 DoTA——面向字节码的轻量级智能合约分析框架。作为首个全面考虑 `hard_xref` 跳转的分析工具，DoTA 通过创新性地结合细粒度控制流分析和污点传播技术，实现了对智能合约中各类 DoS 漏洞的系统性检测。具体而言，我们构建了考虑延迟跳转参数依赖关系的精确控制流模型，并在此基础上开发了高效的循环及递归分析算法，同时引入上下文敏感的污点分析机制来追踪可能污染控制变量的外部输入。通过四个维度的系统性评估（包括检测精度、分析效率、漏洞覆盖率和可扩展性），实验结果表明 DoTA 在各项指标上均显著优于当前最先进的检测工具（包括 Slither、Mythril、eTainter 和 AVVerifier）。特别是在复杂控制流分析和跨合约调用场景下，DoTA 展现出明显的技术优势，其检测准确率达到 88%。这些实证结果充分验证了 DoTA 设计理念的先进性和工程实现的可靠性。

6.1 未来工作

随着区块链技术的快速演进，智能合约的应用场景和平台日益多元化。为了使我们的分析框架能够适应这一发展趋势，未来我们计划从算法细节、多平台适配、多语言支持以及跨链漏洞检测 4 个方面进行拓展，以构建更加全面的智能合约安全保障体系。

1. 算法细节 针对当前依赖 `networkx` 通用图算法库导致的性能瓶颈问题，我们将基于智能合约字节码控制流特有的结构特征（如高度线性化的基本块序列、受限的跳转模式等），设计专用图遍历算法。通过利用字节码控制流图的稀疏性和局部性特征，预期可降低算法的时间复杂度从 $O(n^2)$ 至 $O(n \log n)$ 量级。除了静态分析能力，我们还将加入动态分析的功能：在符号执行方面，我们仅需要将现有的 `opcode_handler` 进行符号化即可；而在模糊测试方面，我们需要设计针对智能合约函数的生成器，从而实现覆盖更多的代码与基本块。此外，我们也将进一步适配旧版本的 `opcode` 模拟执行，以此提高可扩展性。

2. 多区块链平台适配 我们首先关注 EVM 兼容链的扩展。尽管当前系统已能有效分析以太坊网络上的智能合约，但市场上还存在众多 EVM 兼容链，如 BNB Chain、Polygon 和 Avalanche 等。它们在 Gas 模型、区块限制、预编译合约等细节上存在差异。对此，我们计划设计一个适配层，自动根据目标链的特性调整分析参数。例如，BNB Chain 的 Gas 费用较低，而 Polygon 则实现了 EIP-1559 的变体，我们将据此调整 Gas 耗尽类漏洞的检测阈值，使结果更符合实际部署环境。此外，对于 Optimism、

Arbitrum 等 Layer2 方案，我们也将开发专门的分析模块，用于追踪 L1/L2 桥接调用路径，识别可能的跨层漏洞，如状态根挑战失败或延迟攻击。除了 EVM 生态，我们还将支持非 EVM 架构的区块链平台，以适应未来多元化的区块链格局。例如，在 Solana 平台上，我们计划开发 BPF 字节码反编译引擎，通过 LLVM-IR 重构合约控制流图，并结合 Solana 独特的账户模型，设计针对性的安全检测规则。对于 Aptos 与 Sui 等新兴平台，我们将构建基于 Move 语言的字节码分析框架，重点关注资源的生命周期、线性类型限制与权限控制等方面的漏洞。而在 Cosmos 生态中，我们将开发 CosmWasm 合约分析工具，着重分析跨合约调用路径，并识别 IBC 通信协议中存在的潜在安全风险。这些平台之间的执行模型与状态管理机制迥异，将推动我们重新设计底层架构，以实现灵活适配。

3. 智能合约语言的多样性支持 Solidity 之外，Rust 和 Move 等语言正日益成为重要的开发工具。我们计划开发面向 Rust 的安全分析模块，覆盖 Solana 合约及 Polkadot 的 ink! 框架，采用中间层 MIR 对所有权错误、内存越界及不安全调用等问题进行静态检测。针对 Move 语言，我们将专门设计资源追踪引擎，用于验证合约中资源的使用是否存在重复释放或权限绕过的风险，尤其关注其线性类型带来的独特漏洞模式。为实现多语言环境下的一致性分析，我们将开发统一的中间表示（IR）系统，构建 Solidity、Move 与 Rust 等语言间的结构映射关系，统一表示各语言生成的控制流图。这一机制将使得跨语言合约可以用一致的方式进行漏洞检测。更进一步，我们还将设计一种平台无关的漏洞规则描述语言，以声明式方式表达重入、权限缺陷和复杂度攻击等漏洞特征，从而在支持新语言时复用已有规则，降低开发成本。

4. 推进跨链漏洞检测能力的建设 在检测方向上，我们将关注桥接合约中签名验证逻辑的正确性，防范消息重放和信任模型缺陷等问题。同时，构建状态同步追踪系统，用于识别多链状态一致性中的薄弱点，避免出现双花攻击、链回滚等风险。此外，我们还将分析跨链回调机制，评估预言机或中继器在执行回调时可能被利用篡改状态的攻击路径。为提升跨链检测的效果，我们将搭建一个多链模拟环境，集成 EVM、Solana 和 Cosmos 等主流平台的运行时，以支持完整的跨链交互流程测试。我们也计划扩展 W3C 的 Trace Context 标准，使其适应链间消息追踪的需求，便于捕获多链调用链的上下文信息。在此基础上，结合图理论方法，我们将开发跨链交易依赖分析引擎，识别调用路径中的原子性假设是否成立，预警可能的先后序攻击路径。

通过上述算法细节、多平台、多语言和跨链分析的拓展，我们的智能合约安全系统将实现从链内安全走向跨链协同防御的跃升，全面提升区块链应用的安全保障水平，并为构建更加健壮可信的 Web3 基础设施提供坚实支撑。

6.2 行业未来工作与发展趋势

6.2.1 检测漏洞

检测“语言特性导致的漏洞”相对容易，目前已有多种工具可以检测，比如 **slither**[11]、**mythril**[13]、**RNVulDet**[2] 等。但是检测因为逻辑设计不当导致的漏洞，如算法复杂性漏洞，仍然是一个具有挑战性的问题，越来越多的安全从业人员正投身于这一领域的研究。比如 K. Babel 等人提出 **CFF**[28] 基于形式化验证对智能合约的经济安全属性进行自动推理；Z. Li 等人提出的 **ActLifter**[29] 和 **ActCluster** 对 DeFi 的 MEV 活动进行了系统性分析；W. Zhang 等人提出的 **Nyx**[30] 基于符号执行检测智能合约中的抢跑攻击；Y. Duan 提出的 **VetSC**[31] 基于 UI 驱动的程序分析技术进行模型检查，检测 DApps 中的安全性漏洞，比如过期彩票与双重投票；S. Yang 等人提出的 **Hyperion**[32] 通过 LLM 和数据流引导的符号执行揭示了 DApp 前后端逻辑的不一致性。

而导致这一趋势的原因是多样的。首先，智能合约及去中心化应用（DApps）的复杂性不断提高。随着 DeFi、NFT、DAO 和跨链交互等应用的兴起，智能合约不仅承担着资产管理、交易撮合等功能，还涉及复杂的经济博弈与合约间调用。这些复杂逻辑使得漏洞不再局限于传统的语言特性错误，而更多地体现在设计缺陷和合约交互的隐蔽漏洞，如算法复杂性攻击、MEV 滥用、价格操纵等。

其次，传统静态与动态分析工具难以应对高层次的业务逻辑漏洞。像 **Slither** 和 **Mythril** 这样的工具主要依赖静态分析或符号执行来检测已知的模式化漏洞，如重入攻击和整数溢出。但对于涉及多个智能合约、外部数据源（如预言机）、或跨链交互的复杂场景，这些工具往往难以精准建模和分析。例如，套利策略、流动性操控、抢跑攻击等漏洞通常涉及链上交易动态环境，传统工具难以捕捉其完整上下文。

再者，新型智能合约攻击技术的演变推动了检测方法的升级。攻击者不断探索新型的攻击路径，例如利用智能合约的 gas 限制触发拒绝服务（DoS），或通过前端 UI 设计误导用户执行高风险操作（如钓鱼 DApps）。这使得检测逻辑漏洞的工具需要结合程序分析、经济学建模、机器学习、形式化验证等多种技术，以提高漏洞识别能力。

此外，基于大语言模型（LLM）的安全分析正在崛起。如 **Hyperion** 结合数据流分析和 LLM，能够更有效地揭示智能合约和前端交互的不一致性。未来，与 LLM 结合更紧密的安全分析工具有望进一步提升漏洞检测能力，自动推理合约逻辑，识别潜在的业务风险。

综上所述，随着智能合约应用的复杂度提升、传统检测工具的局限性显现，以及

新型攻击手段的涌现，面向逻辑设计缺陷的漏洞检测将成为智能合约安全领域的核心研究方向。未来的工具和方法将更加依赖跨学科融合，结合形式化验证、符号执行、经济学分析和 AI 推理，以提供更全面的安全保障。

6.2.2 检测方法

由于机器学习技术不断发展，越来越多的分析工具开始使用基于机器学习与大语言模型的方法，如 **GPTScan**[33]、**Lanturn**[34]、**PropertyGPT**[35]、**SCVHunter**[36] 和 **Hyperion**[32] 等。这些工具通过学习大量的智能合约数据，提取其中的模式和规律，从而实现对智能合约的自动化分析。基于机器学习和大语言模型的检测方法正在成为未来的发展趋势。

这一趋势的核心原因在于基于机器学习和大语言模型 (LLM) 的检测方法相比传统规则匹配和静态分析方法，具备更强的泛化能力和适应性。传统方法往往依赖于预定义的规则或特征集，难以应对不断变化的攻击模式和新型漏洞。而机器学习方法，特别是基于大语言模型的分析工具，可以通过大规模数据训练，自动学习智能合约中的潜在漏洞模式，即使面对此前未见的攻击方式，也能识别潜在的安全风险。

此外，LLM 能够理解自然语言和代码的深层语义关系，不仅能检测语法级别的错误，还能推理出合约逻辑中的潜在缺陷。相比于传统的静态分析或符号执行方法，这种能力使其更擅长处理复杂的合约逻辑和交互行为，从而提升检测的准确性和覆盖范围。

另一个推动因素是智能合约生态的快速发展，新型 DeFi 协议、跨链桥和 Layer 2 解决方案的兴起使得合约结构愈发复杂。基于 LLM 的检测工具可以不断学习新趋势，自动适应新兴应用场景，从而保持较高的检测有效性。同时，随着开源大语言模型的进步，如 CodeLlama、GPT-4-Turbo 和 StarCoder 等，安全研究者可以微调或定制模型，以满足特定的智能合约安全需求，进一步增强检测工具的实用性。

综上所述，基于机器学习和大语言模型的检测方法因其自动化、泛化能力强、语义理解深刻以及对新兴威胁的适应性，正在成为智能合约安全分析领域未来的发展趋势。

6.2.3 区块链平台

除了经典的比特币和以太坊，越来越多的新型区块链网络和智能合约平台正在迅速崛起，如 Solana、Sui、Avalanche、Polkadot、Binance Smart Chain 等。这些新兴网络在架构设计和技术实现上各具特色，例如 Solana 采用并行交易处理 (Parallel

Processing) 以提升吞吐量, Sui 利用对象模型 (Object-Based Model) 进行高效状态管理, 而 Polkadot 依靠跨链通信 (Interoperability) 促进不同区块链之间的协作。这些创新不仅提供了更高的性能和更低的交易费用, 同时也引入了更加丰富的智能合约开发语言和编程范式, 如 Move、Rust、Huff 等。

然而, 随着区块链生态的不断演进, 智能合约的安全问题也变得日益复杂和严峻。新型区块链的底层执行环境、共识机制和账户模型均与传统 EVM (Ethereum Virtual Machine) 有所不同, 使得在以太坊上行之有效的安全分析方法可能无法直接适用于这些新平台。例如, Solana 采用的基于账户的并行执行模型与以太坊的全局状态机存在显著差异, 这意味着针对 EVM 的传统静态分析工具 (如 Slither) 可能无法精准捕捉 Solana 智能合约的漏洞。此外, 新语言如 Move 和 Sway 采用更严格的安全模型, 虽然减少了一些传统攻击面, 但仍可能存在编译器漏洞、运行时错误或逻辑缺陷, 而这些问题尚未被充分研究。

因此, 针对新型区块链网络的智能合约漏洞检测已成为未来的重要研究方向。研究人员需要开发适配不同链的安全分析工具, 结合形式化验证、动态分析、符号执行等多种方法, 以更精准地识别漏洞。例如, 针对 Move 语言合约的形式化验证工具、适用于 Solana 的并行执行模拟器, 以及支持多链环境的静态/动态分析框架, 均是当前和未来的研究重点。随着 DeFi、NFT、跨链协议的广泛应用, 新型区块链网络上的安全问题将更加突出, 智能合约漏洞检测技术的持续升级, 将成为保障去中心化生态安全的关键。

6.2.4 智能合约语言

而随着智能合约生态的演进, 新的智能合约编程语言不断涌现, 以提高安全性、可扩展性和开发效率。例如, Move 通过线性类型系统防止资产双花, Vyper 通过精简语法减少攻击面, Liquidity 通过函数式编程和形式化验证增强安全性。然而, 这些新兴语言仍然面临诸多安全挑战, 包括编译器漏洞、运行时环境安全性以及开发者使用不当导致的逻辑错误。

一方面, 新语言的成熟度较低, 可能存在尚未被广泛发现的安全漏洞。例如, CVE-2025-27104 和 CVE-2025-27105 等近期披露的 Vyper 漏洞表明, 即使是为安全而设计的语言, 也可能因实现缺陷而引发安全问题。此外, 编译器和虚拟机 (如 MoveVM、Michelson) 的安全性同样需要关注, 一旦底层执行环境存在漏洞, 智能合约的安全性也会受到影响。

另一方面, 开发者对新语言的理解和最佳实践尚不完善, 容易在编码时引入错

误。例如，Move 的资源模型要求严格的资源管理，稍有不慎可能导致意外的访问控制缺陷。Vyper 由于不支持某些 Solidity 语法（如 inline assembly），可能导致开发者误用替代方案，引入逻辑漏洞。Liquidity 依赖 OCaml 生态，而 OCaml 社区在智能合约开发领域的经验有限，使得错误处理和优化仍存在改进空间。

因此，未来的研究方向不仅包括检测传统 EVM 合约的漏洞，还需要探索 Move、Vyper、Liquidity 等新语言的安全风险。针对这些语言，研究者需要开发专门的安全分析工具，如形式化验证框架、静态分析工具和 AI 赋能的安全扫描器，以应对新型智能合约语言带来的安全挑战。

参考文献

- [1] Fortune Business Insights. Blockchain Market Size. Available at: <https://www.fortunebusinessinsights.com/industry-reports/blockchain-market-100072>, Accessed: 2025-08-09.
- [2] P. Qian et al., "Demystifying Random Number in Ethereum Smart Contract: Taxonomy, Vulnerability Identification, and Attack Detection," in IEEE Transactions on Software Engineering, vol. 49, no. 7, pp. 3793-3810, July 2023.
- [3] L. Brent, N. Grech, S. Lagouvardos, B. Scholz, and Y. Smaragdakis, "Ethainter: a smart contract security analyzer for composite vulnerabilities," in Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, London UK: ACM, Jun. 2020, pp. 454–469.
- [4] T. Sun, N. He, J. Xiao, Y. Yue, X. Luo, and H. Wang, "All Your Tokens are Belong to Us: Demystifying Address Verification Vulnerabilities in Solidity Smart Contracts," May 31, 2024, arXiv: arXiv:2405.20561.
- [5] Bo Jiang, Ye Liu, and W. K. Chan. 2018. ContractFuzzer: fuzzing smart contracts for vulnerability detection. In Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18). Association for Computing Machinery, New York, NY, USA, 259–269.
- [6] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. 2022. SMARTIAN: enhancing smart contract fuzzing with static and dynamic data-flow analyses. In Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21). IEEE Press, 227–239.
- [7] Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. Available at: <https://bitcoin.org/bitcoin.pdf>, 2008.
- [8] Wood, G. Ethereum: A Secure Decentralised Generalised Transaction Ledger. Available at: <https://ethereum.github.io/yellowpaper/paper.pdf>, 2014.
- [9] DASP. DASP Top 10. Available at: <https://dasp.co/>.
- [10] DeFiHackLabs. List of DeFi Hacks & POCs. Available at: <https://github.com/SunWeb3Sec/DeFiHackLabs/tree/main>.
- [11] J. Feist, G. Grieco and A. Groce, "Slither: A Static Analysis Framework for Smart Contracts," 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software

- Engineering for Blockchain (WETSEB), Montreal, QC, Canada, 2019, pp. 8-15, doi: 10.1109/WETSEB.2019.00008.
- [12] J. F. Ferreira, P. Cruz, T. Durieux and R. Abreu, "SmartBugs: A Framework to Analyze Solidity Smart Contracts," 2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, VIC, Australia, 2020, pp. 1349-1352.
- [13] CONSENSYS. Mythril: Security Analysis Tool for Ethereum Smart Contracts. Available at: <https://github.com/ConsenSys/mythril>.
- [14] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making Smart Contracts Smarter. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16). Association for Computing Machinery, New York, NY, USA, 254–269.
- [15] OWASP. Smart Contract Security Top 10. Available at: <https://scs.owasp.org/sctop10/>.
- [16] Asem Ghaleb, Julia Rubin, and Karthik Pattabiraman. 2022. ETainter: detecting gas-related vulnerabilities in smart contracts. In Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2022). Association for Computing Machinery, New York, NY, USA, 728–739.
- [17] J. Xu, J. Shen and S. Tan, "DoSChecker: An Efficient and Dedicated Tool for Detecting DoS Vulnerability in Smart Contracts," 2023 International Conference on Data Security and Privacy Protection (DSPP), Xi'an, China, 2023, pp. 22-31.
- [18] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*. <https://arxiv.org/abs/2210.03629>
- [19] Are We There Yet? Unraveling the State-of-the-Art Smart Contract Fuzzers. Available at: <https://arxiv.org/abs/2402.02973>.
- [20] Tai D. Nguyen, Long H. Pham, Jun Sun, Yun Lin, and Quang Tran Minh. 2020. SFuzz: an efficient adaptive fuzzer for solidity smart contracts. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20). Association for Computing Machinery, New York, NY, USA, 778–788.
- [21] M. Rodler et al., "EF $\square$ CF: High Performance Smart Contract Fuzzing for Exploit Generation," 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), Delft, Netherlands, 2023, pp. 449-471, doi: 10.1109/EuroSP57164.2023.00034.

- [22] T. Zhou, K. Liu, L. Li, Z. Liu, J. Klein and T. F. Bissyandé, "SmartGift: Learning to Generate Practical Inputs for Testing Smart Contracts," 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), Luxembourg, 2021, pp. 23-34, doi: 10.1109/ICSME52107.2021.00009.
- [23] Chaofan Shou, Shangyin Tan, and Koushik Sen. 2023. ItyFuzz: Snapshot-Based Fuzzer for Smart Contract. In Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 322–333. <https://doi.org/10.1145/3597926.3598059>
- [24] N. Ruaro, F. Gritti, R. McLaughlin, I. Grishchenko, C. Kruegel, and G. Vigna, "Not your Type! Detecting Storage Collision Vulnerabilities in Ethereum Smart Contracts," in Proceedings 2024 Network and Distributed System Security Symposium, San Diego, CA, USA: Internet Society, 2024.
- [25] Jie, W., Qiu, W., Yang, H. et al. Agent4Vul: multimodal LLM agents for smart contract vulnerability detection. Sci. China Inf. Sci. 68, 160101 (2025). <https://doi.org/10.1007/s11432-024-4402-2>
- [26] Yu, L., Huang, Z., Yuan, H., Cheng, S., Yang, L., Zhang, F., Shen, C., Ma, J., Zhang, J., Lu, J., Zuo, C. (2025). SmartLLaMA-DPO: Reinforced large language model for explainable smart contract vulnerability detection. In Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '25) (Vol. 2, Article ISSTA009). ACM. <https://doi.org/10.1145/3728878>
- [27] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H. (2024). Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. <https://arxiv.org/abs/2402.01030>
- [28] K. Babel, P. Daian, M. Kelkar, and A. Juels, "Clockwork Finance: Automated Analysis of Economic Security in Smart Contracts," in 2023 IEEE Symposium on Security and Privacy (SP), May 2023, pp. 2499–2516.
- [29] Z. Li et al., "Demystifying DeFi MEV Activities in Flashbots Bundle," in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Copenhagen Denmark: ACM, Nov. 2023, pp. 165–179.
- [30] W. Zhang et al., "Nyx: Detecting Exploitable Front-Running Vulnerabilities in Smart Contracts," in 2024 IEEE Symposium on Security and Privacy (SP), May 2024, pp. 2198–2216.

- [31] Y. Duan et al., “Towards Automated Safety Vetting of Smart Contracts in Decentralized Applications,” in Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, Los Angeles CA USA: ACM, Nov. 2022, pp. 921–935.
- [32] S. Yang et al., ”Hyperion: Unveiling DApp Inconsistencies using LLM and Dataflow-Guided Symbolic Execution,” in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 2025, pp. 178-190.
- [33] Y. Sun et al., “GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis,” in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon Portugal: ACM, Apr. 2024, pp. 1–13. doi: 10.1145/3597503.3639117.
- [34] K. Babel, M. Javaheripi, Y. Ji, M. Kelkar, F. Koushanfar, and A. Juels, “Lanturn: Measuring Economic Security of Smart Contracts Through Adaptive Learning,” 2023, 2023/1338. Accessed: Feb. 19, 2025. [Online].
- [35] Y. Liu et al., “PropertyGPT: LLM-driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation,” Dec. 06, 2024.
- [36] F. Luo et al., “SCVHunter: Smart Contract Vulnerability Detection Based on Heterogeneous Graph Attention Network,” in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon Portugal: ACM, Apr. 2024, pp. 1–13.
- [37] S. A. Crosby and D. S. Wallach. Denial of Service via Algorithmic Complexity Attacks. In *Proceedings of the 12nd Security Symposium*, Washington, D.C., 2003. USENIX Association.
- [38] Etherscan: The EVM Explorer. Available at: <https://etherscan.io/>.